

MAY 4, 2026 SYLLABUS — 7 DOMAINS

DATABRICKS CERTIFIED DATA ENGINEER ASSOCIATE

Complete Exam Preparation Guide | 200 Practice Questions | Key Topics Cheat Sheet

200

Practice Questions

7

Exam Domains

45

Real Exam Questions

70%

Pass Mark (~32/45)

QUESTIONS

45 scored MCQs

TIME LIMIT

90 minutes

PASS MARK

~70% (≈32/45 correct)

REGISTRATION FEE

USD 200 + taxes

VALIDITY

2 years

DELIVERY

Online proctored or test centre

#	Domain	Est. %	~Questions	Min Correct (70%)	New in May 2026?
1	Databricks Intelligence Platform	10%	5	4	Core/updated
2	Data Ingestion & Loading	20%	9	6	Lakeflow Connect added
3	Data Transformation & Modeling	18%	8	6	LDP/SDP focus added
4	Working with Lakeflow Jobs	12%	5	4	Data-driven triggers added
5	Implementing CI/CD	10%	5	4	NEW NEW domain
6	Troubleshooting, Monitoring & Optimization	15%	7	5	NEW NEW — Liquid Clustering, Predictive Opt.
7	Governance & Security	15%	6	4	NEW ABAC, Column Masking, DENY added
TOTAL		100%	45	~32	

📅 6-WEEK STUDY PLAN

Week 1: Domain 1 — Delta Lake, Photon, compute types, ACID, time travel, Z-Ordering

Week 2: Domain 2 — Auto Loader, COPY INTO, Lakeflow Connect, JDBC, schema evolution

Week 3: Domain 3 — Medallion architecture, LDP/SDP, PySpark transforms, aggregations, tuning

Week 4: Domain 4 + 5 — Lakeflow Jobs, trigger types, DABs, CI/CD, Databricks CLI NEW

Week 5: Domain 6 + 7 — Spark UI, Liquid Clustering, Predictive Optimization, UC governance, ABAC NEW

Week 6: Full mock exam ×3 using this guide — target 80%+ before sitting the real exam

DOMAIN 1: Databricks Intelligence Platform

Topic	Key Point (1-2 lines)
Lakehouse Architecture	Combines data lake flexibility (open formats, low cost) with data warehouse reliability (ACID, schema, performance) in one platform built on Delta Lake.
Delta Lake	Open-source storage layer adding ACID transactions, schema enforcement, time travel, and scalable metadata on top of Parquet files in cloud storage.
All-Purpose Cluster	Interactive multi-language cluster for notebook development. Expensive if left idle — use for dev/exploration only.
Job Cluster	Spun up for a specific job run, terminated on completion. Most cost-efficient for automated production workloads.
SQL Warehouse	Purpose-built, fully managed compute for SQL and BI queries. Auto-scales, no cluster config needed, billed per DBU consumed.
Serverless Compute	Databricks manages all infrastructure; starts in seconds; scales automatically; no idle cost. Best for variable workloads.
Photon Engine	Databricks-native C++ vectorized query engine accelerating Spark SQL/DataFrame ops without code changes.
Control Plane vs Data Plane	Control plane = Databricks-managed (web app, job scheduler, metadata). Data plane = customer cloud (clusters, storage, data processing).
DBU	Databricks Unit — normalized unit of processing power used for billing. Higher DBR/larger instances = more DBUs per hour.
Unity Catalog	Centralised governance layer providing unified access control, lineage, and auditing for all data assets across workspaces. Hierarchy: Metastore → Catalog → Schema → Table.

DOMAIN 2: Data Ingestion & Loading

Topic	Key Point (1-2 lines)
Auto Loader (cloudFiles)	Incremental file ingestion using <code>.format("cloudFiles")</code> . Tracks processed files via checkpoint. Supports directory listing or file notification mode.
cloudFiles.schemaLocation	Persists inferred/defined schema across runs to prevent drift. Required for production Auto Loader pipelines.
rescuedDataColumn	Captures unexpected/mismatched fields into a JSON column instead of failing. Prevents data loss from schema mismatches.
COPY INTO	Idempotent SQL command loading files from cloud storage into Delta. Tracks loaded files — safe to re-run, no duplicates. Best for smaller file sets.
Lakeflow Connect	Managed connector service for enterprise sources (Salesforce, SAP, JDBC). Handles schema evolution, error recovery, incremental tracking automatically.
Schema Enforcement	Delta Lake rejects writes not conforming to the table schema. Prevents accidental schema drift at the storage layer.
Schema Evolution	<code>cloudFiles.schemaEvolutionMode=addNewColumns</code> auto-adds new source columns to target table without failing pipeline.
JDBC Ingestion	<code>spark.read.jdbc()</code> with url, dbtable, user, password. Use <code>partitionColumn + lowerBound + upperBound + numPartitions</code> for parallel reads.
Auto Loader vs COPY INTO	Auto Loader = Structured Streaming, scales to billions of files, event-driven. COPY INTO = SQL command, simpler, idempotent, best for moderate file volumes.

DOMAIN 3: Data Transformation & Modeling

Topic	Key Point (1-2 lines)
Medallion Architecture	Bronze = raw as-is (append-only). Silver = cleansed/validated/typed. Gold = aggregated/business-ready KPIs. Each layer adds quality.
@dp.table (Streaming Table)	2026 SDP decorator for streaming tables backed by <code>readStream</code> . Processes incrementally. Use for Auto Loader and CDC sources.
@dp.materialized_view	Batch-computed table fully recomputed each run. Use for aggregations and joins with no streaming source (e.g. KPI tables, <code>dim_date</code>).
@dp.temporary_view	Intermediate view within pipeline, not persisted to catalog. Used as staging before CDC flows (SCD Type II).
Data Quality Expectations	<code>@dp.expect</code> = warn only (keep row). <code>@dp.expect_or_drop</code> = remove bad rows. <code>@dp.expect_or_fail</code> = halt pipeline. Choose based on criticality.
SCD Type II	<code>dp.create_streaming_table()</code> + <code>dp.create_auto_cdc_flow()</code> . Active records have <code>__END_AT</code> IS NULL. Old records get <code>__END_AT</code> timestamp.
Broadcast Join	Sends small table to all executors avoiding shuffle. Use <code>broadcast(df)</code> hint or set <code>spark.sql.autoBroadcastJoinThreshold</code> . Best when one side <10MB.
shuffle.partitions	<code>spark.sql.shuffle.partitions</code> (default 200) controls post-shuffle partitions. Reduce for small data (<1GB), increase for large datasets.
dropDuplicates()	Streaming-safe deduplication. <code>ROW_NUMBER()</code> is NOT supported on streams. Use <code>withWatermark()</code> + <code>dropDuplicates()</code> for cross-batch dedup.
approx_count_distinct()	Uses HyperLogLog to estimate distinct count. Much faster than <code>countDistinct()</code> for large datasets with configurable accuracy.

DOMAIN 4: Working with Lakeflow Jobs

Topic	Key Point (1-2 lines)
Lakeflow Jobs	Databricks orchestration service for multi-task DAGs with dependencies, retries, schedules, and alerts. Supports notebook, SQL, pipeline, Python, dbt tasks.
Repair Run	Reruns only failed/skipped tasks without re-executing successful ones. Saves time and compute vs full rerun.
Scheduled Trigger	Cron-based time trigger. Best when data arrives on predictable schedule.
File Arrival Trigger	Fires when new file detected in cloud storage path. Enables event-driven pipelines without polling.
Table Update Trigger	Fires when upstream Delta table is updated. Best for data-driven pipeline chaining when upstream completion time varies.
Conditional Tasks	Tasks can branch based on upstream success/failure. Enables if/else and loop patterns in Lakeflow Jobs DAG.
Task Types	Notebook, Python script, SQL query, Dashboard, Pipeline (SDP), dbt, Spark JAR. Each configured with cluster/warehouse and dependencies.

DOMAIN 5: Implementing CI/CD NEW MAY 2026

Topic	Key Point (1-2 lines)
Databricks Repos	Git integration in workspace UI. Create branches, commit, push, pull, create PRs directly from Databricks. Syncs with GitHub/GitLab/Azure DevOps.
Declarative Automation Bundles (DAB)	Formerly Databricks Asset Bundles. YAML-based IaC for deploying jobs, pipelines, notebooks across dev/test/prod. Root file: databricks.yml.
DAB Targets	Environment-specific overrides (catalog, cluster size) in databricks.yml targets block. Same codebase, different configs per env.
DAB CLI Commands	validate (check YAML syntax), deploy (push resources to workspace), run (trigger execution), destroy (remove deployed resources).
Databricks CLI	Command-line tool for managing workspaces, clusters, jobs, and DABs. Used in CI/CD pipelines for automated deployment.
CI/CD Workflow	Developer commits → PR created → review → merge to main → GitHub Actions/Azure DevOps triggers → bundle deploy → bundle run.

DOMAIN 6: Troubleshooting, Monitoring & Optimization NEW MAY 2026

Topic	Key Point (1-2 lines)
Spark UI — Stages Tab	Shows task-level metrics per stage. Look for long max task times (skew), high shuffle read/write (shuffle bottleneck), high GC time (memory pressure).
Data Skew	Some partitions have far more data than others → few tasks run much longer. Fix: salting, AQE skew join optimization, repartition by different key.
Disk Spill	Executor memory insufficient → intermediate data written to local disk. Fix: increase spark.executor.memory, reduce partition size, increase numPartitions.
Liquid Clustering	Replaces OPTIMIZE + ZORDER. Clusters data by specified columns incrementally without full table rewrite. More flexible than static partitioning.
Predictive Optimization	Databricks auto-runs OPTIMIZE and VACUUM on managed Unity Catalog tables based on usage patterns. No manual scheduling needed.
AQE	Adaptive Query Execution. Dynamically adjusts query plan at runtime — coalesces small shuffle partitions, fixes skew, switches join strategies.
OOM (Out of Memory)	Driver OOM: large collect() or broadcast. Executor OOM: large partitions, excessive caching. Fix: increase memory, reduce data per partition, avoid collect() on large data.
Lakeflow Jobs Run History	Compare current run duration vs historical baseline. Spikes indicate data volume increase, new bottleneck, or resource contention.

DOMAIN 7: Governance & Security EXPANDED MAY 2026

Topic	Key Point (1-2 lines)
Managed vs External Tables	Managed: UC controls lifecycle — DROP TABLE deletes data. External: data persists after DROP TABLE. External uses LOCATION keyword.
GRANT / REVOKE / DENY	GRANT privilege ON object TO principal. REVOKE removes it. DENY explicitly blocks (overrides GRANT even if user has it via group).
UC Privilege Hierarchy	Need USE CATALOG + USE SCHEMA + SELECT to query a table. Privilege inheritance flows down: grant on catalog covers all schemas/tables within.
Column Masking	SQL function applied at query time to transform/hide column values based on querying user's identity. Analysts see masked values (e.g. last 4 digits).
Row-Level Security	Boolean function applied at query time restricting rows visible per user/group. Users see only rows matching their identity filter.
ABAC Policies	Attribute-Based Access Control. Centrally defined UC policies combining row filters and column masks based on user attributes/group membership.

Delta Sharing	Open protocol to share live Delta tables externally without copying data. Recipients use open-source client — no Databricks account needed.
Lakehouse Federation	Query external DBs (MySQL, Snowflake, PostgreSQL) directly from UC via foreign catalogs. No data movement — federated query at runtime.
Audit Logs	Workspace events (access, permissions, runs) delivered as JSON to customer cloud storage (S3/ADLS/GCS). Used for compliance and forensics.
Data Lineage	UC auto-captures column-level lineage across notebooks, jobs, pipelines. View in Catalog Explorer — no code changes required.

✦ Domain 1: Databricks Intelligence Platform

Q1

Which component of the Databricks Data Intelligence Platform is hosted entirely in the control plane?

Domain 1: Databricks Intelligence Platform

- A. Worker node
- B. JDBC data source
- C. Databricks web application ✓
- D. Databricks Filesystem (DBFS)

Answer: C | **Why:** The Databricks web application (UI, job scheduler, metadata store) runs in Databricks' control plane. Worker nodes, JDBC data sources, and data processing all run in the customer's data plane (cloud account).

Q2

A team runs ad-hoc SQL queries and dashboards refreshing every hour using BI tools. Which compute type is most appropriate?

Domain 1: Databricks Intelligence Platform

- A. All-purpose cluster
- B. Job cluster
- C. SQL Warehouse ✓
- D. GPU cluster

Answer: C | **Why:** SQL Warehouses are purpose-built for SQL analytics and BI queries. They auto-scale, require no configuration, and are billed per DBU consumed. All-purpose clusters are for interactive notebook development, not optimised for BI workloads.

Q3

What is a Databricks Unit (DBU)?

Domain 1: Databricks Intelligence Platform

- A. A unit of data storage equal to 1GB
- B. A normalized unit of processing power used for billing on the Databricks platform ✓
- C. A type of cluster node
- D. A measure of network throughput

Answer: B | **Why:** DBU (Databricks Unit) is the normalized pricing unit. The number of DBUs consumed depends on the instance type, DBR version, and workload type. 1 DBU = Databricks running on an i3.xlarge with standard runtime for 1 hour.

Q4

Which Delta Lake feature allows a user to query a table as it existed 7 days ago?

Domain 1: Databricks Intelligence Platform

- A. Schema evolution
- B. Delta cloning
- C. Time travel using VERSION AS OF or TIMESTAMP AS OF ✓
- D. VACUUM with retention period

Answer: C | **Why:** Delta Lake time travel enables querying historical versions: `SELECT * FROM table TIMESTAMP AS OF '2026-05-01'` or `VERSION AS OF 10`. This is powered by the immutable transaction log (`_delta_log`).

Q5

What is the primary purpose of OPTIMIZE in Delta Lake?

Domain 1: Databricks Intelligence Platform

- A. Removes old data files
- B. Compacts many small Parquet files into fewer larger files to improve read performance ✓
- C. Encrypts data at rest
- D. Validates table schema against incoming data

Answer: B | **Why:** OPTIMIZE compacts small Parquet files (created by frequent small writes) into right-sized files (~128MB). This improves read performance by reducing the number of files scanned. Combined with ZORDER BY it also improves data skipping.

Q6

A data engineer needs to run a Spark job nightly. The cluster should not exist outside of the job run to minimise cost. Which cluster type should be used?

Domain 1: Databricks Intelligence Platform

- A. All-purpose cluster kept running 24/7
- B. Job cluster created at job start and terminated on completion ✓**
- C. SQL Warehouse
- D. High-concurrency cluster

Answer: B | **Why:** Job clusters are created specifically for a job run and automatically terminated when done. This eliminates idle compute cost. All-purpose clusters that run 24/7 waste DBUs during off-hours.

Q7

What does VACUUM in Delta Lake do?

Domain 1: Databricks Intelligence Platform

- A. Optimises file layout for query performance
- B. Removes unreferenced data files older than the retention threshold (default 7 days) ✓**
- C. Compresses the transaction log
- D. Refreshes cached query results

Answer: B | **Why:** VACUUM table_name [RETAIN n HOURS] deletes Parquet files no longer referenced by the Delta transaction log. Default retention is 7 days (168 hours). Running VACUUM removes the ability to time-travel before the retention period.

Q8

Which statement correctly describes SHALLOW CLONE vs DEEP CLONE in Delta Lake?

Domain 1: Databricks Intelligence Platform

- A. SHALLOW copies data files; DEEP copies only metadata
- B. SHALLOW copies only metadata and references original files; DEEP copies both metadata and all data files ✓**
- C. Both copy data files but SHALLOW is faster
- D. CLONE is not a Delta Lake feature

Answer: B | **Why:** SHALLOW CLONE copies only the Delta transaction log — the clone references original data files without copying them. It's fast but dependent on source files. DEEP CLONE copies the actual Parquet data files, creating a fully independent copy.

Q9

What storage format does Delta Lake use to store data files?

Domain 1: Databricks Intelligence Platform

- A. ORC
- B. Avro
- C. Parquet ✓**
- D. JSON

Answer: C | **Why:** Delta Lake stores data as Parquet files (columnar format) and adds a transaction log (_delta_log directory containing JSON/checkpoint files) on top to provide ACID guarantees, schema enforcement, and time travel capabilities.

Q10

A company wants to share Delta tables with an external partner. The partner does not use Databricks. Which feature enables this?

Domain 1: Databricks Intelligence Platform

- A. Unity Catalog external tables
- B. Delta Sharing open protocol ✓**
- C. Databricks Repos
- D. DBFS mount points

Answer: B | **Why:** Delta Sharing is an open-source protocol allowing sharing of live Delta tables with external recipients regardless of their platform. Open-source client libraries exist for Python, pandas, Spark, Power BI — no Databricks account required.

Q11

Which Databricks feature enables Change Data Feed on a Delta table?

Domain 1: Databricks Intelligence Platform

- A. ALTER TABLE t SET TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true') ✓**
- B. ENABLE CDF ON TABLE t
- C. SET delta.cdf = true FOR TABLE t
- D. CREATE TABLE t WITH CDF

Answer: A | **Why:** Change Data Feed is enabled with: ALTER TABLE table_name SET TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true'). Once enabled, Delta captures row-level changes (insert/update_preimage/update_postimage/delete) readable via table_changes().

Q12

What is the Unity Catalog three-level namespace for referencing a table?

Domain 1: Databricks Intelligence Platform

- A. database.table
- B. catalog.database
- C. **catalog.schema.table** ✓
- D. metastore.catalog.table

Answer: C | **Why:** Unity Catalog uses a mandatory three-level namespace: catalog.schema.table. This replaces the legacy two-level hive_metastore.schema.table. All UC objects must be addressed with this full three-part path.

Q13

Which compute option starts the fastest and requires zero infrastructure configuration from the user?

Domain 1: Databricks Intelligence Platform

- A. All-purpose cluster with auto-scaling
- B. Job cluster with spot instances
- C. **Serverless compute** ✓
- D. SQL Warehouse Classic

Answer: C | **Why:** Serverless compute starts in seconds (vs minutes for classic clusters) because Databricks pre-provisions warm pools. Users write code and run it immediately — no instance types, node counts, or cluster policies to configure.

Q14

A data engineer runs DESCRIBE HISTORY my_table. What does this return?

Domain 1: Databricks Intelligence Platform

- A. Column names and data types
- B. **A log of all operations (writes, schema changes, optimizations) with version, timestamp, user, and operation details** ✓
- C. The current table statistics
- D. A list of all files in the table

Answer: B | **Why:** DESCRIBE HISTORY table_name returns the full audit trail of every transaction: version number, timestamp, userId, userName, operation (WRITE, OPTIMIZE, VACUUM, MERGE etc.), operationParameters, and cluster ID.

Q15

What is the purpose of Z-Ordering in Delta Lake?

Domain 1: Databricks Intelligence Platform

- A. Reduces file count by compacting small files
- B. **Co-locates related data in files based on column values to improve data skipping during queries** ✓
- C. Encrypts data using a Z-order cipher
- D. Sorts data alphabetically for faster string searches

Answer: B | **Why:** OPTIMIZE ... ZORDER BY column rearranges data so rows with similar column values are stored together. Delta Lake's data skipping can then skip entire files that don't match a filter on the Z-Ordered column, dramatically reducing I/O.

Q16

Which statement about the Databricks Lakehouse architecture is correct?

Domain 1: Databricks Intelligence Platform

- A. It requires separate storage for structured and unstructured data
- B. It stores all data in a proprietary format owned by Databricks
- C. **It unifies data lake storage (open formats, cloud object storage) with data warehouse capabilities (ACID, schema, performance) in a single platform** ✓
- D. It only supports batch processing, not streaming

Answer: C | **Why:** The Lakehouse eliminates the traditional two-tier architecture (data lake + data warehouse). Data lives in open formats (Parquet/Delta) on cloud object storage with full ACID, schema enforcement, and warehouse-level query performance.

Q17

A Delta table has 10,000 small files from frequent streaming writes. Which command resolves this?

Domain 1: Databricks Intelligence Platform

- A. VACUUM my_table
- B. REFRESH TABLE my_table
- C. **OPTIMIZE my_table** ✓
- D. ANALYZE TABLE my_table

Answer: C | **Why:** OPTIMIZE compacts the 10,000 small files into fewer large files (~128MB each). This improves read performance by reducing file listing overhead and enabling more efficient Parquet scanning. Combine with ZORDER BY to also improve data skipping.

Q18

Which Delta Lake feature prevents writes that violate a column's data type or schema definition?

Domain 1: Databricks Intelligence Platform

- A. Schema evolution
- B. Schema enforcement ✓
- C. VACUUM
- D. Time travel

Answer: B | **Why:** Schema enforcement (schema validation) rejects any write operation where the data's schema doesn't match the table's defined schema. Delta Lake will throw an AnalysisException, preventing schema drift from polluting the table.

Q19

What is Predictive Optimization in Databricks?

Domain 1: Databricks Intelligence Platform

- A. An ML model that predicts query runtimes
- B. A feature that automatically runs OPTIMIZE and VACUUM on managed Unity Catalog tables based on observed usage patterns ✓
- C. A query plan cache that predicts the best join strategy
- D. A feature that pre-fetches data based on predicted queries

Answer: B | **Why:** Predictive Optimization (enabled on Unity Catalog managed tables) automatically schedules and runs OPTIMIZE and VACUUM based on table access patterns, eliminating manual maintenance. Available on Databricks with Unity Catalog.

Q20

A data engineer wants to enable auto-compaction of small files after each write to a Delta table. Which table property enables this?

Domain 1: Databricks Intelligence Platform

- A. delta.autoOptimize.autoCompact = true ✓
- B. delta.optimize.autoRun = true
- C. delta.compact.onWrite = true
- D. delta.autoOptimize.vacuum = true

Answer: A | **Why:** Setting delta.autoOptimize.autoCompact=true enables automatic compaction after writes. Combined with delta.autoOptimize.optimizeWrite=true (produces right-sized files during write), this minimises small file accumulation without manual OPTIMIZE runs.

Q21

Which Databricks component manages authentication, access control policies, and audit logging across all workspaces in an organisation?

Domain 1: Databricks Intelligence Platform

- A. Databricks Repos
- B. Unity Catalog metastore ✓
- C. Photon engine
- D. Job scheduler

Answer: B | **Why:** The Unity Catalog metastore is the centralised governance layer. It manages access control (GRANT/REVOKE), data lineage, and audit log generation for all data assets across multiple workspaces attached to the same metastore.

Q22

What is the default VACUUM retention period in Delta Lake and why does it exist?

Domain 1: Databricks Intelligence Platform

- A. 1 day — to allow same-day recovery
- B. 7 days — to allow long-running transactions and reader queries to complete before old files are deleted ✓
- C. 30 days — for monthly compliance audits
- D. 0 days — files are deleted immediately after each transaction

Answer: B | **Why:** The default 7-day (168-hour) retention ensures any long-running Spark jobs, streaming queries, or time-travel queries still referencing older file versions can complete before those files are deleted. Setting retention below 7 days with VACUUM is blocked by default.

Q23

A data engineer needs to revert a Delta table to version 5 after a bad write at version 6. Which command achieves this?

Domain 1: Databricks Intelligence Platform

- A. ROLLBACK TABLE my_table TO VERSION 5
- B. RESTORE TABLE my_table TO VERSION AS OF 5 ✓
- C. UNDO TABLE my_table VERSION 5
- D. REVERT TABLE my_table TO VERSION 5

Answer: B | **Why:** RESTORE TABLE table_name TO VERSION AS OF version creates a new transaction log entry that effectively sets the table state back to the specified version. The original bad write history is preserved (not deleted) but the table now reflects version 5's data.

Q24

Which statement about Liquid Clustering in Delta Lake is correct?

Domain 1: Databricks Intelligence Platform

- A. It physically partitions data by column values like PARTITION BY
- B. It replaces OPTIMIZE + ZORDER with incremental, flexible clustering that doesn't require full table rewrites ✓
- C. It compresses data using a liquid compression algorithm
- D. It enables liquid-speed query execution via Photon

Answer: B | **Why:** Liquid Clustering is the modern replacement for ZORDER. It clusters data incrementally (only new/changed files are reclustered) and supports changing cluster keys without rewriting the full table. More flexible and efficient than static partitioning + ZORDER.

Q25

In Databricks architecture, where does Spark code actually execute?

Domain 1: Databricks Intelligence Platform

- A. In the Databricks control plane (Databricks cloud)
- B. In the customer's data plane (customer's cloud account on customer-managed VMs) ✓
- C. In a shared multi-tenant environment
- D. On the user's local machine via Databricks Connect

Answer: B | **Why:** Spark code executes on worker nodes in the customer's data plane — VMs running in the customer's cloud account (AWS, Azure, or GCP). The control plane only manages scheduling, metadata, and coordination. Customer data never leaves their cloud.

Q26

Which cluster pool feature reduces job startup time?

Domain 1: Databricks Intelligence Platform

- A. Auto-scaling groups
- B. Pre-provisioned idle instances in a cluster pool that new jobs can claim immediately ✓
- C. Spot instance pricing
- D. Photon engine pre-warming

Answer: B | **Why:** Cluster pools maintain a set of pre-provisioned, idle VM instances. When a job cluster or all-purpose cluster needs to start, it claims instances from the pool instantly instead of waiting for cloud VM provisioning (which takes 2-5 minutes).

Q27

What happens to data files when a managed Delta table is dropped with DROP TABLE?

Domain 1: Databricks Intelligence Platform

- A. Files are archived to DBFS for 30 days
- B. Files are moved to a recycle bin
- C. Both metadata and underlying Parquet data files are permanently deleted by Unity Catalog ✓
- D. Only metadata is removed; data files remain at their storage location

Answer: C | **Why:** For managed tables, Unity Catalog controls the full data lifecycle. DROP TABLE removes both the UC metadata (catalog entry) AND permanently deletes the physical Parquet data files from cloud storage. External tables only lose metadata.

Q28

A data engineering team needs compute for machine learning model training requiring GPU acceleration. Which cluster type supports this?

Domain 1: Databricks Intelligence Platform

- A. SQL Warehouse
- B. Serverless pipeline compute
- C. GPU-enabled all-purpose or job cluster with GPU instance types ✓
- D. Standard job cluster

Answer: C | **Why:** GPU workloads (ML training, deep learning) require clusters configured with GPU instance types (e.g. p3.xlarge on AWS, NC-series on Azure). SQL Warehouses and serverless pipeline compute do not support GPU instances — they're for SQL/data engineering workloads.

✦ Domain 2: Data Ingestion & Loading

Q29

Which Spark format string activates Auto Loader for incremental file ingestion?

Domain 2: Data Ingestion & Loading

- A. delta
- B. parquet
- C. cloudFiles ✓
- D. autoLoader

Answer: C | **Why:** Auto Loader is activated by `.format('cloudFiles')`. The cloudFiles format instructs Spark to use Databricks' Auto Loader framework, which incrementally processes new files using checkpointing to track what has already been processed.

Q30

A data engineer runs `COPY INTO my_table FROM 's3://bucket/data/' FILEFORMAT = PARQUET` a second time with no new files. What happens?

Domain 2: Data Ingestion & Loading

- A. An error is thrown for duplicate execution
- B. All files are reloaded creating duplicate rows
- C. No files are loaded — `COPY INTO` tracks loaded files in the Delta log and skips them ✓
- D. The target table is truncated and reloaded

Answer: C | **Why:** `COPY INTO` is idempotent by design. It records which files have been processed in the Delta transaction log. On subsequent runs it skips already-loaded files. This makes it safe to re-run without creating duplicates.

Q31

What is the primary difference between Auto Loader file notification mode and directory listing mode?

Domain 2: Data Ingestion & Loading

- A. File notification mode is slower
- B. Directory listing scans all files each run (expensive at scale); file notification uses cloud event queues (SQS/Event Grid) to detect new files instantly without full directory scan ✓
- C. File notification requires a paid addon
- D. Directory listing only works with CSV files

Answer: B | **Why:** At scale (millions of files), directory listing becomes prohibitively expensive. File notification mode (`cloudFiles.useNotifications=true`) uses cloud-native event notifications to detect new files immediately without listing — dramatically more efficient for large buckets.

Q32

A data engineer sets `cloudFiles.inferColumnTypes = false` in Auto Loader. What is the effect?

Domain 2: Data Ingestion & Loading

- A. Auto Loader will fail if any type mismatch is found
- B. All columns will be read as `StringType` unless an explicit schema is provided ✓
- C. Auto Loader will only process integer columns
- D. Schema inference is disabled and Auto Loader will use the last known schema

Answer: B | **Why:** With `inferColumnTypes=false`, Auto Loader reads all values as strings. An explicit schema should be provided using `.schema()`. This is recommended in production to prevent schema drift from unexpected values (e.g. '123abc' in a column expected to be integer).

Q33

What does the `rescuedDataColumn` option `'_rescued_data'` do in Auto Loader?

Domain 2: Data Ingestion & Loading

- A. Creates a separate quarantine table
- B. Deletes rows with unexpected columns
- C. Captures any column that doesn't match the defined schema into a single JSON column, preserving all data ✓
- D. Sends an alert when unexpected data is found

Answer: C | **Why:** `rescuedDataColumn` option specifies a column name (e.g. `'_rescued_data'`) where Auto Loader stores any field that doesn't fit the schema (wrong type, extra column) as a JSON string. Data is never silently dropped — it's rescued for inspection.

Q34

Which Auto Loader option automatically adds new source columns to the target Delta table schema?

Domain 2: Data Ingestion & Loading

- A. `cloudFiles.allowSchemaChanges=true`
- B. `cloudFiles.schemaEvolutionMode=addNewColumns` ✓
- C. `cloudFiles.inferColumnTypes=true`
- D. `cloudFiles.schemaLocation=/path`

Answer: B | **Why:** `cloudFiles.schemaEvolutionMode=addNewColumns` tells Auto Loader to detect new columns in source files and automatically add them to the target Delta table schema. Without this, new columns either fail or land in the rescued data column.

Q35

When should COPY INTO be preferred over Auto Loader?

Domain 2: Data Ingestion & Loading

- A. When ingesting from Kafka streams
- B. When source has billions of files and requires event-driven processing
- C. When loading a moderate number of files from cloud storage in a simple, idempotent SQL-based approach ✓**
- D. When real-time latency is required

Answer: C | **Why:** COPY INTO is ideal for simpler, SQL-based batch loads of moderate file volumes. It's idempotent and easy to use. Auto Loader is preferred for very large file volumes (millions+), event-driven ingestion, or when Structured Streaming is required.

Q36

Which PySpark API reads data from a PostgreSQL database?

Domain 2: Data Ingestion & Loading

- A. spark.read.postgres()
- B. spark.read.jdbc(url=..., table=..., properties={...}) ✓**
- C. spark.read.sql('SELECT * FROM postgres_table')
- D. spark.read.rdbms()

Answer: B | **Why:** spark.read.jdbc() is the standard PySpark JDBC reader. Required parameters: url (JDBC connection string), dbtable (table or subquery), and properties dict with user/password/driver. This works for any JDBC-compliant database.

Q37

A data engineer needs to read a large table from MySQL in parallel using JDBC. Which parameters must be set?

Domain 2: Data Ingestion & Loading

- A. parallelism and fetchSize
- B. partitionColumn, lowerBound, upperBound, and numPartitions ✓**
- C. shardKey and numShards
- D. splitColumn and batchSize

Answer: B | **Why:** For parallel JDBC reads, set partitionColumn (a numeric/date column), lowerBound (min value), upperBound (max value), and numPartitions (number of parallel tasks). Spark creates numPartitions sub-queries splitting the range evenly.

Q38

What is Lakeflow Connect used for in Databricks?

Domain 2: Data Ingestion & Loading

- A. Connecting Databricks notebooks to Git repositories
- B. A managed connector service for ingesting data from enterprise sources (Salesforce, SAP, Workday, databases) into Unity Catalog tables ✓**
- C. Connecting SQL Warehouses to BI tools
- D. Managing network connectivity between workspaces

Answer: B | **Why:** Lakeflow Connect is Databricks' managed ingestion service with pre-built connectors for enterprise sources. It handles authentication, incremental syncing, schema mapping, error recovery, and UC governance automatically.

Q39

A pipeline uses Auto Loader to ingest JSON files with nested arrays. Which PySpark function extracts array elements into separate rows?

Domain 2: Data Ingestion & Loading

- A. F.flatten()
- B. F.explode() ✓**
- C. F.unnest()
- D. F.expand()

Answer: B | **Why:** F.explode(array_column) creates one row per element in the array. For nested JSON with arrays, read the file, then apply .withColumn('item', F.explode('items_array')) to flatten the array into individual rows.

Q40

What does cloudFiles.maxFilesPerTrigger control in Auto Loader?

Domain 2: Data Ingestion & Loading

- A. Maximum file size in bytes per trigger
- B. Maximum number of new files processed per micro-batch trigger in triggered streaming mode ✓**
- C. Maximum number of files in the source directory
- D. Maximum files per executor

Answer: B | **Why:** cloudFiles.maxFilesPerTrigger limits how many new files are processed in each micro-batch when running Auto Loader in triggered mode. This controls throughput and prevents memory pressure from loading too many files at once.

Q41

A data engineer needs to ingest Salesforce data into Databricks daily. Which is the most appropriate approach?

Domain 2: Data Ingestion & Loading

- A. Custom Python script using the Salesforce REST API in a notebook
- B. COPY INTO with Salesforce URL
- C. Lakeflow Connect with a Salesforce managed connector ✓
- D. spark.read.jdbc() with Salesforce JDBC driver

Answer: C | **Why:** Lakeflow Connect provides a managed Salesforce connector handling OAuth authentication, incremental syncing (only changed records), schema mapping, and UC governance. Custom REST API scripts require significantly more engineering and maintenance.

Q42

Which COPY INTO syntax correctly loads CSV files with a header from an ADLS path?

Domain 2: Data Ingestion & Loading

- A. COPY INTO my_table FROM 'abfss://container@account.dfs.core.windows.net/path/' FORMAT = CSV OPTIONS (header = true)
- B. COPY INTO my_table FROM 'abfss://...' FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'true') ✓
- C. LOAD INTO my_table FROM 'abfss://...' FILEFORMAT = CSV
- D. INSERT INTO my_table SELECT * FROM 'abfss://...'

Answer: B | **Why:** COPY INTO table FROM 'path' FILEFORMAT = CSV FORMAT_OPTIONS ('header' = 'true', 'inferSchema' = 'true') is the correct syntax. FORMAT_OPTIONS pass file-format-specific options. COPY_OPTIONS handles loading behaviour (e.g. 'force' = 'true' to reload).

Q43

What is the purpose of the _metadata column available when reading files with Auto Loader or spark.read?

Domain 2: Data Ingestion & Loading

- A. Stores the file's encryption key
- B. Contains file-level metadata: file_path, file_name, file_size, file_modification_time — accessible as a struct column ✓
- C. Tracks which user loaded the file
- D. Contains row-level checksums for validation

Answer: B | **Why:** _metadata is a Spark built-in hidden metadata column available when reading files. It's a struct with fields: file_path, file_name, file_size, file_modification_time, file_block_start, file_block_length. Access with F.col('_metadata.file_path').

Q44

A team ingests data from multiple sources: IoT device files (millions/day), Salesforce CRM (daily batch), and a legacy Oracle DB (hourly). Which ingestion method matches each source correctly?

Domain 2: Data Ingestion & Loading

- A. COPY INTO for all three
- B. Auto Loader for IoT, Lakeflow Connect for Salesforce, spark.read.jdbc() for Oracle ✓
- C. JDBC for all three
- D. Lakeflow Connect for all three

Answer: B | **Why:** Auto Loader handles high-volume file ingestion efficiently. Lakeflow Connect manages enterprise SaaS sources like Salesforce. JDBC is appropriate for relational databases like Oracle, especially with scheduled notebooks orchestrated by Lakeflow Jobs.

Q45

What is schema enforcement in the context of Delta Lake writes?

Domain 2: Data Ingestion & Loading

- A. Automatically adds new columns to the schema
- B. Rejects any write where the data schema doesn't match the table's registered schema, preventing silent data corruption ✓
- C. Enforces column naming conventions
- D. Validates data against business rules

Answer: B | **Why:** Schema enforcement is Delta Lake's write-time validation. If a DataFrame being written has extra columns, missing columns, or wrong data types compared to the table schema, the write is rejected with an AnalysisException. This prevents schema drift.

Q46

A data engineer needs to ingest a one-time historical load of 50GB of Parquet files from S3 into a Delta table. Which method is most appropriate?

Domain 2: Data Ingestion & Loading

- A. Auto Loader in continuous mode
- B. COPY INTO ✓
- C. spark.readStream.format('cloudFiles')
- D. Lakeflow Connect

Answer: B | **Why:** For a one-time or occasional bulk load, COPY INTO is the simplest and most appropriate. Auto Loader is designed for ongoing incremental ingestion, not one-time bulk loads. COPY INTO handles the 50GB efficiently and is idempotent.

Q47

Which Auto Loader checkpoint option stores streaming state to enable exactly-once processing across restarts?

Domain 2: Data Ingestion & Loading

- A. `cloudFiles.schemaLocation`
- B. **`cloudFiles.checkpointLocation` (set as `.option('checkpointLocation', path)`) ✓**
- C. `cloudFiles.stateLocation`
- D. `cloudFiles.recoveryPath`

Answer: B | **Why:** The checkpoint location (set via `.option('checkpointLocation', '/path/to/checkpoint')`) stores Structured Streaming state: which files have been processed, offsets, and micro-batch metadata. On restart, Auto Loader resumes from the last checkpoint.

Q48

Which statement about COPY INTO's file tracking is correct?

Domain 2: Data Ingestion & Loading

- A. COPY INTO uses a separate checkpoint table
- B. **COPY INTO records processed files in the Delta transaction log of the target table ✓**
- C. COPY INTO requires a manifest file listing processed files
- D. COPY INTO uses the source file's modification timestamp to detect already-loaded files

Answer: B | **Why:** COPY INTO stores its file tracking state in the Delta transaction log of the target table. When you run SHOW HISTORY on the target table, you can see COPY INTO operations and which files were loaded. This is why COPY INTO must target a Delta table.

Q49

What format does Lakeflow Connect use to ingest semi-structured JSON data with deeply nested fields?

Domain 2: Data Ingestion & Loading

- A. It only supports flat JSON
- B. **It automatically flattens nested JSON into columns using dot notation and can handle arrays with `array_max_level` configuration ✓**
- C. It converts JSON to CSV before loading
- D. It requires manual schema definition for all nested fields

Answer: B | **Why:** Lakeflow Connect handles nested JSON by mapping nested objects to dot-notation column names (e.g. `address.city`) and arrays to array-typed columns. The exact behaviour depends on the connector configuration and schema inference settings.

Q50

A data engineer configures Auto Loader with `cloudFiles.useNotifications=true` but the pipeline fails to detect new files. What is the most likely cause?

Domain 2: Data Ingestion & Loading

- A. The source bucket is empty
- B. **The cloud notification service (SQS/Event Grid) is not configured or lacks permissions to send events to Auto Loader ✓**
- C. The files are in the wrong format
- D. The `schemaLocation` is missing

Answer: B | **Why:** File notification mode requires cloud-side configuration: an SQS queue (AWS) or Event Grid subscription (Azure) must be set up to receive new-object events from the storage bucket and Auto Loader needs permission to read from that notification queue.

Q51

Which ingestion pattern is correct for loading new records from a source table into Delta incrementally using a high-watermark approach?

Domain 2: Data Ingestion & Loading

- A. `SELECT * FROM source`
- B. Use COPY INTO with `FORCE=true` to reload all data
- C. **Read source with a filter on a timestamp/sequence column greater than the last processed value, then MERGE INTO or append to target ✓**
- D. TRUNCATE target and reload from source daily

Answer: C | **Why:** High-watermark incremental loading: track the last processed timestamp/sequence, read only new records from source (`WHERE updated_at > last_watermark`), then MERGE INTO target (upsert) or INSERT INTO (append-only). Store the new watermark for the next run.

Q52

What is the advantage of using Unity Catalog governed ingestion (landing data directly into UC tables) vs landing in DBFS first?

Domain 2: Data Ingestion & Loading

- A. DBFS is slower than UC tables
- B. UC provides access control, lineage tracking, and audit logging for data from the moment it lands — DBFS has none of these governance capabilities ✓
- C. UC tables use less storage
- D. DBFS does not support Delta format

Answer: B | **Why:** Landing data directly into Unity Catalog tables means governance (access control, lineage, auditing) applies immediately. Data in DBFS has no UC governance — anyone with workspace access can read DBFS files, which violates data governance best practices.

Q53

Which file discovery mode should be used when Auto Loader source bucket has strict IAM policies preventing LIST operations?

Domain 2: Data Ingestion & Loading

- A. Directory listing mode — it doesn't use LIST
- B. File notification mode — uses cloud event notifications instead of LIST operations ✓
- C. Manifest mode — uses a pre-generated file list
- D. Schema inference mode

Answer: B | **Why:** File notification mode avoids LIST operations entirely by subscribing to cloud storage events (S3 event notifications, Azure Event Grid). This works even when IAM policies restrict s3:ListBucket, as only s3:GetObject and SQS permissions are needed.

Q54

A COPY INTO command targets a table that already has 1 million rows. New source files with 50,000 rows are added. What is the result after running COPY INTO again?

Domain 2: Data Ingestion & Loading

- A. The table is truncated and 50,000 rows are loaded
- B. 1,050,000 rows — existing rows preserved, only new files' rows appended
- C. The command fails because files already exist
- D. 50,000 rows — only new files loaded, existing rows unchanged (append-only) ✓

Answer: D | **Why:** COPY INTO appends rows from new (previously unloaded) files only. Existing 1 million rows are preserved. The 50,000 new rows are appended, resulting in 1,050,000 total. Option C (50,000 rows) is wrong because it implies existing rows are replaced.

Q55

Which statement about Lakeflow Connect standard connectors vs managed connectors is correct?

Domain 2: Data Ingestion & Loading

- A. Standard connectors are open-source; managed connectors are proprietary
- B. Standard connectors are Databricks-built with full support; managed connectors are partner-built and certified through the Databricks marketplace ✓
- C. They are identical — 'standard' and 'managed' are interchangeable terms
- D. Managed connectors require separate licensing

Answer: B | **Why:** Lakeflow Connect standard connectors are built and fully supported by Databricks. Managed connectors are partner-built connectors available through the Databricks marketplace. Both are production-ready but have different support ownership.

Q56

What is the correct Auto Loader syntax to read CSV files from a Volume path with an explicit schema?

Domain 2: Data Ingestion & Loading

- A. `spark.read.format('csv').load('/Volumes/...')`
- B. `spark.readStream.format('cloudFiles').option('cloudFiles.format','csv').schema(schema).load('/Volumes/...')` ✓
- C. `spark.readStream.format('autoLoader').option('format','csv').load('/Volumes/...')`
- D. `spark.read.autoLoader('/Volumes/...')`

Answer: B | **Why:** Auto Loader requires: `spark.readStream` (streaming API), `.format('cloudFiles')` (Auto Loader format), `.option('cloudFiles.format', 'source_format')` (actual file format), `.option('cloudFiles.schemaLocation', path)` (schema tracking), optionally `.schema(schema)`, then `.load(source_path)`.

✦ Domain 3: Data Transformation & Modeling

Q57

In the Medallion Architecture, which layer stores data in its raw, unmodified form as received from source systems?

Domain 3: Data Transformation & Modeling

- A. Gold layer
- B. Silver layer
- C. Bronze layer ✓
- D. Platinum layer

Answer: C | **Why:** Bronze is the raw ingestion layer. Data lands exactly as received from the source — no transformations, no cleansing. It's append-only and serves as the immutable source of truth for reprocessing silver and gold if business rules change.

Q58

Which SQL command creates a new Delta table only if one doesn't already exist with that name?

Domain 3: Data Transformation & Modeling

- A. CREATE OR REPLACE TABLE t (id INT, name STRING)
- B. CREATE TABLE IF NOT EXISTS t (id INT, name STRING) ✓
- C. CREATE NEW TABLE t (id INT, name STRING)
- D. CREATE TABLE t (id INT, name STRING) ON DUPLICATE IGNORE

Answer: B | **Why:** CREATE TABLE IF NOT EXISTS silently skips creation if the table already exists. CREATE OR REPLACE TABLE drops and recreates the table (losing all existing data). Use IF NOT EXISTS when you want idempotent table creation without destroying data.

Q59

Which SQL command appends a new row to an existing Delta table?

Domain 3: Data Transformation & Modeling

- A. UPDATE my_table VALUES ('a1', 6, 9.4)
- B. APPEND INTO my_table VALUES ('a1', 6, 9.4)
- C. INSERT INTO my_table VALUES ('a1', 6, 9.4) ✓
- D. ADD TO my_table VALUES ('a1', 6, 9.4)

Answer: C | **Why:** INSERT INTO table_name VALUES (val1, val2, ...) is the standard SQL DML command to append rows to a table. For DataFrames, use df.write.mode('append').saveAsTable('table_name') or df.insertInto('table_name').

Q60

What are the correct PySpark aggregation functions to compute total revenue and unique invoice count by billing date?

Domain 3: Data Transformation & Modeling

- A. groupBy('billing_date').agg(sum('amount'), sum('billing_id'))
- B. groupBy('billing_date').agg(sum('amount').alias('total'), count_distinct('billing_id').alias('invoices')) ✓
- C. groupBy('billing_date').agg(col('amount'), count('billing_id'))
- D. groupBy('billing_date').agg(sum('amount'), count_distinct('patient_id'))

Answer: B | **Why:** sum() computes total revenue. count_distinct('billing_id') counts unique invoices (not patients). sum('billing_id') would sum ID numbers (wrong). count('billing_id') would count all rows including duplicates (wrong if billing_id can repeat).

Q61

Which MERGE INTO statement correctly upserts data into a Delta table?

Domain 3: Data Transformation & Modeling

- A. UPSERT INTO target USING source ON target.id = source.id
- B. MERGE INTO target USING source ON target.id = source.id WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT * ✓
- C. UPDATE OR INSERT INTO target FROM source WHERE target.id = source.id
- D. SYNC target WITH source ON target.id = source.id

Answer: B | **Why:** MERGE INTO is Delta Lake's upsert syntax. ON defines the match condition. WHEN MATCHED handles updates/deletes for existing rows. WHEN NOT MATCHED handles inserts for new rows. Multiple WHEN clauses can be chained with different conditions.

Q62

Why is ROW_NUMBER() window function not supported on a streaming DataFrame?

Domain 3: Data Transformation & Modeling

- A. ROW_NUMBER is deprecated in Spark 3+
- B. ROW_NUMBER requires seeing all rows to assign ranks, which is impossible in an infinite stream processed as micro-batches ✓
- C. ROW_NUMBER only works with SQL, not PySpark
- D. ROW_NUMBER requires a sorted Delta table as input

Answer: B | **Why:** Structured Streaming processes data in incremental micro-batches. ROW_NUMBER() is a non-time-based window function that requires a global view of all rows to assign correct ranks — fundamentally incompatible with the streaming micro-batch model.

Q63

Which streaming-safe method replaces ROW_NUMBER() for deduplication on a streaming DataFrame?

Domain 3: Data Transformation & Modeling

- A. rank()
- B. dense_rank()
- C. dropDuplicates(['key_column']) ✓
- D. distinct()

Answer: C | **Why:** dropDuplicates(['key_column']) is the streaming-safe deduplication method. For cross-batch deduplication, combine with .withWatermark('timestamp_col', 'duration') which defines how long Spark tracks seen keys before expiring state.

Q64

A data engineer needs to combine two DataFrames with identical schemas keeping ALL rows including duplicates. Which operation is correct?

Domain 3: Data Transformation & Modeling

- A. df1.join(df2, how='inner')
- B. df1.union(df2) ✓
- C. df1.intersect(df2)
- D. df1.except(df2)

Answer: B | **Why:** df1.union(df2) combines all rows from both DataFrames including duplicates (equivalent to SQL UNION ALL). df1.unionByName(df2) does the same but matches by column name. Both preserve all rows — no deduplication occurs.

Q65

What does spark.sql.shuffle.partitions = 200 mean and when should it be reduced?

Domain 3: Data Transformation & Modeling

- A. Maximum number of files written to storage; reduce when storage is limited
- B. Number of partitions after shuffle operations (join/groupBy); reduce to 8-16 for small datasets (<1GB) to avoid hundreds of tiny tasks ✓
- C. Maximum memory per shuffle; reduce when OOM errors occur
- D. Number of parallel reads; reduce for small datasets

Answer: B | **Why:** spark.sql.shuffle.partitions (default 200) sets post-shuffle partition count. For small datasets (under 1GB), 200 partitions creates excessive overhead with tiny tasks. Rule of thumb: target ~128MB per partition. For 100MB dataset, 1-2 partitions is sufficient.

Q66

Which PySpark function renames a column?

Domain 3: Data Transformation & Modeling

- A. df.rename('old','new')
- B. df.withColumnRenamed('old_name','new_name') ✓
- C. df.aliasColumn('old','new')
- D. df.changeColumn('old','new')

Answer: B | **Why:** df.withColumnRenamed('old_name', 'new_name') renames a single column and returns a new DataFrame. Chain multiple calls for multiple renames. Alternatively, use df.toDF(*new_names_list) to rename all columns at once.

Q67

Which PySpark function adds a new computed column to a DataFrame?

Domain 3: Data Transformation & Modeling

- A. df.addColumn('col', expr)
- B. df.withColumn('col_name', expression) ✓
- C. df.append('col_name', expression)
- D. df.transform('col_name', expression)

Answer: B | **Why:** df.withColumn('new_col', expression) adds or replaces a column. It returns a new immutable DataFrame. The expression can be any Column expression: F.col(), F.lit(), arithmetic, string functions, etc.

Q68

Which PySpark method filters rows where sales > 1000?

Domain 3: Data Transformation & Modeling

- A. df.where('sales > 1000')
- B. df.filter(F.col('sales') > 1000)
- C. df.query('sales > 1000')
- D. Both A and B are correct ✓

Answer: D | **Why:** df.filter() and df.where() are aliases — both accept the same condition syntax: SQL string ('sales > 1000'), Column expression (F.col('sales') > 1000), or DataFrame column (df['sales'] > 1000). All produce identical results.

Q69

What is a broadcast join and when should it be used?

Domain 3: Data Transformation & Modeling

- A. A join that broadcasts all rows to all users
- B. A hint that sends a small DataFrame to all executor nodes, eliminating shuffle. Use when one table is small enough to fit in executor memory (~10MB default threshold) ✓
- C. A join that broadcasts only null rows
- D. A join optimised for broadcasting results to dashboards

Answer: B | **Why:** A broadcast join (`df1.join(broadcast(df2), key)`) copies the smaller DataFrame to all executors. No shuffle of the larger DataFrame occurs. This dramatically speeds up joins where one side is small. Configure `spark.sql.autoBroadcastJoinThreshold` to adjust the auto-broadcast size limit.

Q70

Which aggregate function estimates distinct count efficiently for large datasets without exact computation?

Domain 3: Data Transformation & Modeling

- A. `F.countDistinct()`
- B. **`F.approx_count_distinct(col, rsd=0.05)`** ✓
- C. `F.estimated_distinct()`
- D. `F.hyperloglog_count()`

Answer: B | **Why:** `F.approx_count_distinct()` uses HyperLogLog probabilistic algorithm. The `rsd` parameter (relative standard deviation, default 0.05 = 5% error) controls accuracy vs performance. For billion-row datasets, it's 10-100x faster than exact `countDistinct()`.

Q71

Which LDP/SDP decorator should be used for a Gold layer KPI table that reads from fact_sales using `dp.read()` (batch)?

Domain 3: Data Transformation & Modeling

- A. `@dp.table`
- B. `@dp.streaming_table`
- C. **`@dp.materialized_view`** ✓
- D. `@dp.view`

Answer: C | **Why:** `@dp.materialized_view` is correct for batch-computed tables using `dp.read()` (not `readStream`). `@dp.table` requires a streaming source (`readStream`). The KPI table is recomputed fully each pipeline run from the fact table — a classic materialized view pattern.

Q72

In Lakeflow SDP, which expectation drops rows that violate the rule without halting the pipeline?

Domain 3: Data Transformation & Modeling

- A. `@dp.expect`
- B. `@dp.expect_or_fail`
- C. **`@dp.expect_or_drop`** ✓
- D. `@dp.expect_warn`

Answer: C | **Why:** `@dp.expect_or_drop` removes rows violating the constraint from the output table. The pipeline continues with clean rows. Violation counts are tracked in pipeline metrics. Use for data that is invalid and should not propagate downstream.

Q73

Which SDP functions implement SCD Type II (Slowly Changing Dimension Type 2)?

Domain 3: Data Transformation & Modeling

- A. `@dp.table()` + `dp.read_stream()`
- B. **`dp.create_streaming_table()` + `dp.create_auto_cdc_flow(stored_as_scd_type=2)`** ✓
- C. `@dp.materialized_view()` + `dp.merge()`
- D. `@dp.table()` + `dp.apply_scd2()`

Answer: B | **Why:** SCD Type II in 2026 Lakeflow SDP: `dp.create_streaming_table()` defines the target dimension table, `dp.create_auto_cdc_flow(stored_as_scd_type=2)` configures CDC logic with keys, sequence_by column, and SCD type 2 behaviour (adds `__START_AT`/`__END_AT` columns).

Q74

In an SCD Type II dimension table managed by `create_auto_cdc_flow`, how are currently active records identified?

Domain 3: Data Transformation & Modeling

- A. `is_current = True` column
- B. **`__END_AT IS NULL`** ✓
- C. `__START_AT = current_timestamp()`
- D. `scd_status = 'ACTIVE'`

Answer: B | **Why:** Lakeflow SDP automatically manages `__START_AT` and `__END_AT` system columns. Active/current records have `__END_AT IS NULL`. When a record changes, the old row gets `__END_AT` set to the change timestamp, and a new row is inserted with `__END_AT = NULL`.

Q75

What is the correct PySpark syntax to split a string column 'full_name' into first and last name?

Domain 3: Data Transformation & Modeling

- A. `F.substring('full_name', ' ')`
- B. **`F.split(F.col('full_name'), ' ')` ✓**
- C. `F.parse('full_name', ' ')`
- D. `F.tokenize('full_name', ' ')`

Answer: B | **Why:** `F.split(column, pattern)` splits a string by a regex pattern, returning an `ArrayType` column. To access elements: `F.split(F.col('full_name'),' ')[0]` for first name, `[1]` for last. Or use `F.split().getItem(0)` syntax.

Q76

Which PySpark function converts a `StringType` date column '2026-05-15' to `DateType`?

Domain 3: Data Transformation & Modeling

- A. `F.date_parse('col', 'yyyy-MM-dd')`
- B. **`F.to_date(F.col('date_str'), 'yyyy-MM-dd')` ✓**
- C. `F.cast_date('col', 'yyyy-MM-dd')`
- D. `F.str_to_date('col', 'yyyy-MM-dd')`

Answer: B | **Why:** `F.to_date(column, format)` converts string to `DateType`. The format string uses Java date pattern (yyyy=year, MM=month, dd=day). Returns null for unparseable values (PERMISSIVE mode). Alternatively: `df.withColumn('date', F.col('date_str').cast('date'))`

Q77

What does `F.coalesce(col1, col2, col3)` return?

Domain 3: Data Transformation & Modeling

- A. Merges three DataFrames into one
- B. **Returns the first non-NULL value among the specified columns ✓**
- C. Returns the last non-NULL value
- D. Counts non-null values across the columns

Answer: B | **Why:** `F.coalesce(col1, col2, ...)` is the PySpark equivalent of SQL COALESCE — returns the first non-NULL value from left to right. Useful for filling nulls from a priority list of columns: `F.coalesce(F.col('addr1'), F.col('addr2'), F.lit('Unknown'))`.

Q78

What is the difference between `repartition(n)` and `coalesce(n)` in PySpark?

Domain 3: Data Transformation & Modeling

- A. They are identical
- B. **`repartition` performs a full shuffle and can increase or decrease partitions; `coalesce` only decreases partitions with minimal data movement (no full shuffle) ✓**
- C. `coalesce` is for columns; `repartition` is for rows
- D. `repartition` is only for streaming; `coalesce` is for batch

Answer: B | **Why:** `repartition(n)` performs a full shuffle — can increase OR decrease partitions, redistributes evenly, used to fix skew. `coalesce(n)` only decreases partitions by combining existing ones with minimal shuffle — cheaper but can't increase partition count or fix skew.

Q79

Which DDL command in Spark SQL creates a Delta table without specifying column definitions, deriving schema from an existing table?

Domain 3: Data Transformation & Modeling

- A. **`CREATE TABLE new_table LIKE source_table` ✓**
- B. `CREATE TABLE new_table AS SELECT * FROM source_table WHERE 1=0`
- C. `CREATE TABLE new_table SCHEMA LIKE source_table`
- D. `CREATE TABLE new_table FROM source_table`

Answer: A | **Why:** `CREATE TABLE new_table LIKE source_table` creates an empty table with the same schema and table properties (including partitioning) as `source_table` but no data. `CREATE TABLE AS SELECT` copies data too.

Q80

A data engineer wants to apply data quality rules to a Silver table where NULL order_id should drop the row but invalid ship_mode should just log a warning. Which expectations are correct?

Domain 3: Data Transformation & Modeling

- A. @dp.expect_or_fail for both
- B. @dp.expect_or_drop('order_id_not_null', 'order_id IS NOT NULL') and @dp.expect('ship_mode_valid', '...') ✓
- C. @dp.expect for both
- D. @dp.expect_or_drop for both

Answer: B | **Why:** @dp.expect_or_drop removes rows with NULL order_id (critical — can't process without a key). @dp.expect for ship_mode logs violations as metrics but keeps the row (non-critical attribute — better to have the row with a warning than lose the order).

Q81

What does df.describe() return in PySpark?

Domain 3: Data Transformation & Modeling

- A. Column names and data types
- B. Summary statistics (count, mean, stddev, min, max) for numeric and string columns ✓
- C. Row count only
- D. Column nullability information

Answer: B | **Why:** df.describe() returns count, mean, stddev, min, and max for numeric columns, and count, min (lex), max (lex) for string columns. df.summary() returns more percentiles (25%, 50%, 75%). Both are useful for quick data profiling.

Q82

Which PySpark operation removes rows where order_id is NULL?

Domain 3: Data Transformation & Modeling

- A. df.dropna(subset=['order_id'])
- B. df.filter(F.col('order_id').isNotNull())
- C. df.na.drop(subset=['order_id'])
- D. All of the above are correct ✓

Answer: D | **Why:** All three are equivalent ways to remove rows with NULL order_id: df.dropna(subset=['order_id']), df.na.drop(subset=['order_id']) (alias), and df.filter(F.col('order_id').isNotNull()). They produce identical results.

Q83

When should a Gold layer object be created as a view rather than a materialized view or table?

Domain 3: Data Transformation & Modeling

- A. When query results change frequently and materializing would be stale
- B. When a lightweight, always-current logical abstraction is needed that doesn't store data — appropriate for simple column renames or filters used by BI tools ✓
- C. Views are always better than materialized views
- D. When data needs to persist across sessions

Answer: B | **Why:** Views are non-materialising — they execute the underlying query at read time. Use views when you need always-current results, the query is fast, and you don't want to store data. Use materialized views when the query is expensive and pre-computing improves BI query performance.

✦ Domain 4: Working with Lakeflow Jobs

Q84

What is the primary function of Lakeflow Jobs in Databricks?

Domain 4: Working with Lakeflow Jobs

- A. Running SQL queries on Delta tables
- B. Orchestrating multi-task data pipelines with dependencies, retries, schedules, and monitoring ✓
- C. Ingesting data from cloud storage
- D. Managing Unity Catalog permissions

Answer: B | **Why:** Lakeflow Jobs (formerly Databricks Workflows) is the orchestration engine. It defines multi-task DAGs where each task can be a notebook, SQL query, Python script, pipeline, or dbt model. Tasks have dependencies, retry policies, and trigger configurations.

Q85

A Lakeflow Job has 3 tasks: A → B → C. Task B fails. After fixing the issue, what is the most efficient way to rerun?

Domain 4: Working with Lakeflow Jobs

- A. Delete the job and recreate it
- B. Rerun the entire job from Task A
- C. Use Repair Run to rerun only Task B and C, preserving Task A's successful output ✓
- D. Manually trigger Task B and C separately

Answer: C | **Why:** Repair Run reruns only failed/skipped tasks (B and C) while preserving outputs from successful tasks (A). This saves compute time and avoids unnecessary reprocessing. Task A's output remains available for Task B to use.

Q86

Which Lakeflow Jobs trigger type fires when a new file arrives in a cloud storage path?

Domain 4: Working with Lakeflow Jobs

- A. Scheduled trigger
- B. File arrival trigger ✓
- C. Table update trigger
- D. Manual trigger

Answer: B | **Why:** File arrival trigger monitors a specified cloud storage path and automatically starts a job run when a new file is detected. This enables event-driven pipelines that react to data availability rather than running on fixed schedules.

Q87

Which trigger type is most appropriate when a downstream pipeline should run as soon as an upstream Delta table is updated?

Domain 4: Working with Lakeflow Jobs

- A. Scheduled trigger every 5 minutes
- B. File arrival trigger on the upstream table's storage path
- C. Table update trigger on the upstream Delta table ✓
- D. Webhook trigger from the upstream job

Answer: C | **Why:** Table update trigger watches a specified Delta table and fires the downstream job when new data is committed. This is data-driven — the job runs when data is ready, eliminating polling and handling variable upstream completion times.

Q88

What is the difference between a time-based trigger and a data-driven trigger in Lakeflow Jobs?

Domain 4: Working with Lakeflow Jobs

- A. Time-based triggers are free; data-driven triggers cost extra
- B. Time-based (scheduled) triggers fire at fixed intervals regardless of data availability; data-driven (file arrival/table update) triggers fire when data arrives, reducing unnecessary runs ✓
- C. Time-based triggers only support daily schedules
- D. Data-driven triggers require a paid tier

Answer: B | **Why:** Scheduled triggers run at fixed cron intervals even if no new data exists — potentially wasteful. File arrival and table update triggers fire only when actual data changes occur, making them more efficient and responsive for data-dependent pipelines.

Q89

A Lakeflow Job needs to send a different notification based on whether a task succeeds or fails. Which feature enables this?

Domain 4: Working with Lakeflow Jobs

- A. Post-task scripts
- B. Conditional tasks with on_success and on_failure branches ✓
- C. Email notifications in task settings
- D. Webhook tasks

Answer: B | **Why:** Lakeflow Jobs supports conditional branching. Tasks can specify different downstream paths based on the outcome (success/failure/custom exit code) of upstream tasks. This enables if/else workflows: success → dashboard refresh, failure → alert + diagnostic notebook.

Q90

How is a maximum retry count configured for a Lakeflow Jobs task?

Domain 4: Working with Lakeflow Jobs

- A. By wrapping the task code in a try/except loop
- B. By setting max_retries in the task configuration with optional min_retry_interval_millis ✓
- C. By adding a retry task downstream
- D. By enabling the auto_retry cluster setting

Answer: B | **Why:** In Lakeflow Jobs task configuration, max_retries specifies how many automatic retry attempts occur on failure. Combined with min_retry_interval_millis (delay between retries), this handles transient failures (network blips, cloud throttling) without manual intervention.

Q91

Which task type in Lakeflow Jobs runs a Lakeflow SDP pipeline as part of a larger workflow?

Domain 4: Working with Lakeflow Jobs

- A. Python script task
- B. Notebook task with pipeline code
- C. Pipeline task ✓
- D. SQL task

Answer: C | **Why:** The Pipeline task type specifically runs a Lakeflow Spark Declarative Pipeline. It waits for the pipeline run to complete and can trigger a full refresh or incremental run. This allows SDP pipelines to be chained with other task types in a workflow DAG.

Q92

A data team wants a job to run every Monday at 8 AM UTC. Which trigger configuration is correct?

Domain 4: Working with Lakeflow Jobs

- A. schedule: every_monday_8am
- B. cron: '0 8 * * MON' with timezone: UTC ✓
- C. time_trigger: MON:08:00:UTC
- D. schedule_type: weekly, day: MONDAY, time: 08:00

Answer: B | **Why:** Lakeflow Jobs uses standard cron syntax: '0 8 * * MON' means minute=0, hour=8, any day-of-month, any month, Monday. The timezone is specified separately. Databricks also accepts natural language in the UI which translates to cron internally.

Q93

What does the Lakeflow Jobs DAG view show?

Domain 4: Working with Lakeflow Jobs

- A. The data lineage of Delta tables
- B. The task dependency graph showing which tasks depend on which, current run status of each task, and execution order ✓
- C. The Spark execution plan for each task
- D. The cluster configuration for each task

Answer: B | **Why:** The DAG view in Lakeflow Jobs UI shows all tasks as nodes with dependency arrows. Each node shows its current status (running/success/failed/skipped) with colour coding. This makes it easy to spot upstream blockers and understand pipeline structure.

Q94

A data engineer wants task C to run only if task A succeeds AND task B fails. Which Lakeflow Jobs feature supports this?

Domain 4: Working with Lakeflow Jobs

- A. Task dependencies with run_if conditions ✓
- B. Sequential task ordering
- C. Timeout-based task skipping
- D. Parallel task groups

Answer: A | **Why:** Lakeflow Jobs supports run_if conditions on task dependencies. Each dependency can specify 'ALL_SUCCESS', 'AT_LEAST_ONE_SUCCESS', 'NONE_FAILED', 'ALL_DONE' etc. Complex logic (A succeeds AND B fails) can be implemented using these dependency conditions.

Q95

A Lakeflow Job runs nightly at 2 AM. The source data sometimes arrives late (2:30 AM). What trigger type better handles this data availability uncertainty?

Domain 4: Working with Lakeflow Jobs

- A. Move the schedule to 3 AM and hope data arrives
- B. Use file arrival or table update trigger that fires when data actually arrives regardless of clock time ✓
- C. Add a 2-hour timeout
- D. Use continuous mode

Answer: B | **Why:** Data-driven triggers (file arrival/table update) eliminate scheduling uncertainty. The job fires when data is actually available, not at a fixed clock time. This prevents both missing data (job runs before data arrives) and unnecessary waiting.

Q96

Which Lakeflow Jobs feature allows viewing historical run durations to identify performance regressions?

Domain 4: Working with Lakeflow Jobs

- A. Spark UI history
- B. Lakeflow Jobs run history view — shows all past runs with duration, status, and comparison to historical baseline ✓
- C. Delta table audit logs
- D. Databricks billing dashboard

Answer: B | **Why:** The Lakeflow Jobs run history view displays all past job runs with duration, status, start time, and triggered-by. This allows engineers to compare current run times against historical averages, identifying when a job suddenly takes longer (indicating data growth or performance regression).

Q97

A Lakeflow Job task runs a Python notebook that calls an external API. The API occasionally times out. Which configuration handles this gracefully?

Domain 4: Working with Lakeflow Jobs

- A. Set max_retries=3 and min_retry_interval_millis=60000 to retry 3 times with 1-minute delays ✓**
- B. Wrap the API call in infinite retry loop in the notebook
- C. Set the task timeout to 0 (unlimited)
- D. Use a SQL task instead

Answer: A | **Why:** max_retries=3 with a delay (min_retry_interval_millis=60000 = 1 minute) allows the task to automatically retry 3 times after transient API timeouts. The delay gives the external API time to recover. This is the proper platform-level retry solution vs. custom retry logic in code.

Q98

In the context of Lakeflow Jobs task graph, what does 'ALL_SUCCESS' dependency condition mean?

Domain 4: Working with Lakeflow Jobs

- A. The task runs if at least one upstream task succeeds
- B. The task runs only if ALL upstream tasks succeed ✓**
- C. The task always runs regardless of upstream status
- D. The task runs if all upstream tasks are done (success or failure)

Answer: B | **Why:** ALL_SUCCESS (default) means the task runs only when every task it depends on completes successfully. If any upstream task fails or is skipped, the dependent task is also skipped. This ensures downstream tasks don't run on bad/incomplete data.

Q99

A data pipeline job has 10 tasks. Task 3 fails due to a transient error. After 3 automatic retries it still fails. What happens to tasks 4-10?

Domain 4: Working with Lakeflow Jobs

- A. Tasks 4-10 run anyway using Task 3's last partial output
- B. Tasks 4-10 are skipped because their upstream dependency (Task 3) failed ✓**
- C. Tasks 4-10 are queued and run after manual intervention
- D. The entire job restarts from Task 1

Answer: B | **Why:** In a DAG with ALL_SUCCESS dependencies, tasks downstream of a failed task are automatically skipped. This prevents downstream tasks from running on incomplete or corrupted data. The job run ends with status FAILED, and Repair Run can be used to fix and rerun.

✦ Domain 5: Implementing CI/CD

Q100

What is the root configuration file for a Declarative Automation Bundle (formerly DAB)?

Domain 5: Implementing CI/CD

- A. bundle.json
- B. config.yaml
- C. databricks.yml ✓**
- D. settings.toml

Answer: C | **Why:** databricks.yml is the mandatory root configuration file for a Declarative Automation Bundle (DAB). It defines the bundle name, workspace host, targets (dev/staging/prod), and includes references to resource definition files (jobs, pipelines).

Q101

What does 'databricks bundle deploy --target prod' do?

Domain 5: Implementing CI/CD

- A. Runs all jobs in the prod environment
- B. Validates YAML syntax only
- C. Uploads bundle assets and creates/updates Databricks resources (jobs, pipelines) in the prod target workspace ✓**
- D. Destroys all prod resources and redeploys from scratch

Answer: C | **Why:** bundle deploy uploads code assets (notebooks, scripts) to the workspace and creates or updates Databricks resources (jobs, pipelines, clusters) as defined in the bundle YAML for the specified target (prod). It does NOT run the jobs — use bundle run for that.

Q102

How do Declarative Automation Bundles handle environment-specific configurations (different cluster sizes for dev vs prod)?

Domain 5: Implementing CI/CD

- A. Maintain separate YAML files per environment
- B. Use variable definitions with default values in bundle root and override them in per-target blocks in databricks.yml ✓**
- C. Use Git branches per environment with hardcoded values
- D. Manually edit YAML before each deployment

Answer: B | **Why:** DABs use a variables block for configurable values with defaults, and targets blocks for environment-specific overrides. The same codebase is deployed everywhere; only variable values differ. This is the recommended DRY (Don't Repeat Yourself) approach.

Q103

Which Databricks CLI command validates a DAB's YAML configuration without deploying anything?

Domain 5: Implementing CI/CD

- A. databricks bundle check
- B. databricks bundle validate ✓**
- C. databricks bundle test
- D. databricks bundle lint

Answer: B | **Why:** databricks bundle validate checks all YAML files in the bundle for syntax errors, invalid configurations, undefined variables, and logical issues — without deploying any resources. Always run validate before deploy in CI/CD pipelines.

Q104

A team wants Databricks jobs to deploy automatically when code is merged to the main branch. Which approach achieves this?

Domain 5: Implementing CI/CD

- A. Set a file arrival trigger on the Git repository
- B. Configure a Lakeflow Jobs table update trigger
- C. Integrate DABs with a CI/CD tool (GitHub Actions, Azure DevOps) that runs 'bundle deploy' on merge to main ✓**
- D. Use Databricks Repos auto-sync feature

Answer: C | **Why:** The standard CI/CD pattern: GitHub Actions/Azure DevOps detects merge to main → runs 'databricks bundle validate' → runs 'databricks bundle deploy --target prod'. This automates deployment without manual intervention and ensures only reviewed code reaches production.

Q105

In Databricks Repos, which operation creates an isolated copy of the codebase for a new feature without affecting the main branch?

Domain 5: Implementing CI/CD

- A. Commit
- B. Push
- C. Create branch ✓**
- D. Fork

Answer: C | **Why:** Creating a branch creates an isolated copy of the code at a specific commit point. Developers work on the branch (adding features, fixing bugs) without affecting main/production code. Changes are reviewed via PR before merging back to main.

Q106

What is the purpose of 'databricks bundle run'?

Domain 5: Implementing CI/CD

- A. Validates the bundle YAML syntax
- B. Triggers execution of a specific job or pipeline defined in the bundle ✓**
- C. Deploys the bundle to the workspace
- D. Removes the bundle from the workspace

Answer: B | **Why:** databricks bundle run triggers a run of the specified job or pipeline resource defined in the bundle. It can optionally wait for completion (--no-wait for async) and returns exit status — useful as the final step in CI/CD after deploy.

Q107

A data engineer uses databricks bundle destroy. What is the effect?

Domain 5: Implementing CI/CD

- A. Deletes all local bundle YAML files
- B. Removes all Databricks resources (jobs, pipelines) that were deployed by this bundle from the target workspace, leaving local files intact ✓**
- C. Reverts the workspace to the previous bundle version
- D. Deletes the connected Git repository

Answer: B | **Why:** databricks bundle destroy removes only the workspace resources that were created by bundle deploy — jobs, pipelines, cluster policies defined in the bundle. Local files on disk are unchanged. Use with caution in production environments.

Q108

Which statement correctly describes Databricks Repos vs classic workspace notebooks for code management?

Domain 5: Implementing CI/CD

- A. Repos and workspace notebooks are identical
- B. Workspace notebooks are stored in Databricks-proprietary storage with no Git integration; Repos sync directly with Git repositories enabling version control, branching, and collaborative development ✓
- C. Repos only support Python; workspace notebooks support all languages
- D. Repos require a premium tier license

Answer: B | **Why:** Classic workspace notebooks are stored in Databricks' internal storage without native Git integration (manual export/import for versioning). Repos provide first-class Git integration: create branches, commit, push, pull, create PRs — all from the Databricks UI or CLI.

Q109

What is Databricks Connect and what development workflow does it enable?

Domain 5: Implementing CI/CD

- A. A feature that connects two Databricks workspaces for data sharing
- B. An SDK that allows writing Spark code locally in any IDE (VS Code, PyCharm) that executes on a remote Databricks cluster ✓
- C. A connector for linking Databricks to external data sources
- D. A feature for connecting SQL Warehouses to BI tools

Answer: B | **Why:** Databricks Connect (dbconnect) is an SDK that routes local Spark code execution to a remote Databricks cluster. Developers can write and debug PySpark code in their preferred IDE with full IntelliSense, unit testing, and debugger support, while actual processing runs on the cluster.

Q110

In a DAB's databricks.yml, where are environment-specific resource overrides specified?

Domain 5: Implementing CI/CD

- A. In a separate override.yml file
- B. In the targets block of databricks.yml, where each target (dev/prod) can override variables and resource properties ✓
- C. In environment variables on the deployment server
- D. In a separate JSON config file per environment

Answer: B | **Why:** The targets block in databricks.yml defines environment-specific settings: target dev { variables { catalog: dev_catalog, cluster_size: Small } } and target prod { variables { catalog: prod_catalog, cluster_size: Large } }. Resources reference these variables.

Q111

Which CI/CD best practice ensures the same code runs in all environments (dev, staging, prod)?

Domain 5: Implementing CI/CD

- A. Copy-paste code between environments with manual edits
- B. Use DABs with environment-specific variable overrides so the same bundle codebase is deployed to all targets with only configuration differing ✓
- C. Use separate Git branches for each environment with different hardcoded values
- D. Maintain separate Databricks workspaces with manually synchronised notebooks

Answer: B | **Why:** DABs enforce the 'promote same codebase' principle. Business logic (notebooks, pipeline code) is identical across environments. Only infrastructure configuration (catalog names, cluster sizes, schedule frequencies) differs via target variable overrides.

Q112

A developer commits changes to a feature branch in Databricks Repos. What should the next step be in a proper CI/CD workflow?

Domain 5: Implementing CI/CD

- A. Directly deploy to production
- B. Create a pull request for code review, run automated tests, then merge to main after approval ✓
- C. Delete the branch
- D. Push directly to main branch

Answer: B | **Why:** Proper CI/CD: commit to feature branch → create PR → automated tests run (bundle validate, unit tests) → code review by peers → approval → merge to main → automated deployment pipeline triggers. Never push untested code directly to main/production.

Q113

Which Databricks CLI command lists all deployed bundles and their resources in the current workspace?

Domain 5: Implementing CI/CD

- A. databricks bundle list
- B. databricks jobs list (for jobs) and databricks pipelines list (for pipelines) ✓**
- C. databricks workspace list
- D. databricks bundle status

Answer: B | **Why:** The Databricks CLI doesn't have a single 'bundle list' command. Resources deployed by bundles are standard workspace resources. Use 'databricks jobs list' to see jobs, 'databricks pipelines list' to see pipelines. 'databricks bundle validate' shows what would be deployed.

Q114

What is the recommended folder structure for a Declarative Automation Bundle?

Domain 5: Implementing CI/CD

- A. All YAML and code in a single databricks.yml file
- B. databricks.yml at root + resources/ folder for job/pipeline YAML definitions + src/ folder for notebooks and Python source files ✓**
- C. A single notebooks/ folder with all code
- D. Separate repos for each job definition

Answer: B | **Why:** DAB best practice: databricks.yml (root config) + resources/ (job_definition.yml, pipeline_definition.yml) + src/ (notebooks/, Python scripts). This separation keeps infrastructure config separate from business logic and is more maintainable at scale.

✈ Domain 6: Troubleshooting, Monitoring & Optimization

Q115

A Spark job shows multiple tasks with much longer execution times than others in the Spark UI Stages tab. What does this indicate?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. The cluster is undersized
- B. Data skew — some partitions have far more data than others, causing a few tasks to run much longer ✓**
- C. The network connection is unstable
- D. The Delta table needs OPTIMIZE

Answer: B | **Why:** Task duration imbalance (most tasks complete in 2s, a few take 200s) indicates data skew — uneven data distribution across partitions. Fixes: AQE skew join optimization (automatic), salting the skewed key, repartition by a different column, or OPTIMIZE ZORDER on the join key.

Q116

What does high 'Shuffle Read' size in the Spark UI Stages tab indicate?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. Too many input files
- B. Large data exchange between executors during join/groupBy operations — potentially a performance bottleneck ✓**
- C. Inefficient storage format
- D. Network bandwidth exceeded

Answer: B | **Why:** Large shuffle read means significant data movement between executors. This occurs during joins, groupBy, and sort operations. Reduce with: broadcast join for small tables, increase shuffle partitions to reduce per-partition size, or filter data before shuffling.

Q117

What is disk spill in Spark and what causes it?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. Writing final output to cloud storage
- B. When in-memory data for an operation (sort/join/aggregation) exceeds executor memory, Spark writes overflow to local disk — causing severe performance degradation ✓**
- C. Checkpoint data written to DBFS
- D. Audit log overflow

Answer: B | **Why:** Disk spill occurs when executor heap is insufficient for in-memory computation. Spark serialises excess data to local disk (spill to disk in Spark UI metrics). This is 10-100x slower than in-memory processing. Fix: increase spark.executor.memory, reduce partition size, or increase numPartitions.

Q118

What is Liquid Clustering in Delta Lake and what does it replace?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. A data compression algorithm replacing ZSTD
- B. A replacement for OPTIMIZE + ZORDER BY that incrementally clusters data without full table rewrites, with flexible cluster key changes ✓**
- C. A replacement for VACUUM
- D. A new table format replacing Parquet

Answer: B | **Why:** Liquid Clustering replaces the OPTIMIZE + ZORDER BY workflow. It runs CLUSTER BY column in CREATE TABLE, then OPTIMIZE reclusters only files that need it (not full rewrites). Cluster keys can be changed without rewriting the full table. More flexible than static partitioning.

Q119

What is Predictive Optimization and which tables does it apply to?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. A query cache that predicts hot data
- B. A Databricks feature that automatically runs OPTIMIZE and VACUUM on Unity Catalog managed tables based on observed write/read patterns ✓**
- C. A cost predictor for Databricks jobs
- D. A feature that predicts query execution time

Answer: B | **Why:** Predictive Optimization monitors table usage (write frequency, read patterns, file sizes) and automatically schedules OPTIMIZE and VACUUM — eliminating manual maintenance scripts. It applies to Unity Catalog managed Delta tables and is enabled at the catalog or schema level.

Q120

A cluster fails to start with error 'Library conflict: package X version 1.0 is incompatible with package Y'. What should the data engineer do?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. Increase cluster memory
- B. Check the cluster's installed libraries for version conflicts — use a compatible version or isolate libraries using separate clusters or init scripts ✓**
- C. Restart the cluster without any changes
- D. Increase the number of worker nodes

Answer: B | **Why:** Library conflicts occur when installed packages have incompatible dependencies. Solutions: check which libraries conflict (databricks/cluster libraries tab), pin compatible versions, use a cluster-specific init script, or isolate conflicting workloads to separate clusters with different library sets.

Q121

What does high GC (Garbage Collection) time in the Spark UI Executor metrics indicate?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. The network is slow
- B. JVM memory pressure — the JVM spends excessive time on garbage collection, usually indicating insufficient executor memory or too many object allocations ✓**
- C. Disk I/O bottleneck
- D. CPU is overloaded

Answer: B | **Why:** High GC time means the JVM can't keep up with object creation/destruction — typically caused by insufficient spark.executor.memory. Objects can't be promoted to old generation fast enough, causing frequent GC pauses. Fix: increase executor memory or reduce cached data.

Q122

A data engineer notices a job that ran in 30 minutes last week now takes 90 minutes with the same code. The Lakeflow Jobs run history shows this started 3 days ago. What should be investigated first?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. Code was changed accidentally
- B. Check if data volume increased 3 days ago (new data source, upstream changes), then check for new skew or spill in Spark UI ✓**
- C. The cluster was resized
- D. The SQL Warehouse was paused

Answer: B | **Why:** Duration regression tracked to a specific date usually means data volume changed (new upstream source, backfill, schema change) or a new data pattern introduces skew. Check: row count comparison (before vs after), Spark UI for new spill/skew, upstream pipeline changes 3 days ago.

Q123

Which Spark configuration enables dynamic optimisation of query plans at runtime based on actual data statistics?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. spark.sql.dynamic.optimization=true
- B. spark.sql.adaptive.enabled=true (Adaptive Query Execution) ✓
- C. spark.sql.runtime.optimize=true
- D. spark.sql.statistics.dynamic=true

Answer: B | **Why:** AQE (spark.sql.adaptive.enabled=true, default in Spark 3+) collects runtime statistics mid-query and dynamically adjusts: coalesces small shuffle partitions, skew join optimization, and switches join strategies. This often eliminates the need for manual tuning.

Q124

A data engineer has a Delta table with 100,000 small files after 6 months of streaming writes. Query performance is slow. Which sequence of commands best resolves this?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. VACUUM then OPTIMIZE
- B. OPTIMIZE (optionally with ZORDER BY query columns), then VACUUM to clean up old files ✓
- C. REFRESH TABLE then ANALYZE TABLE
- D. CLONE the table to a new location

Answer: B | **Why:** OPTIMIZE first compacts the small files into right-sized files (and applies ZORDER if specified). VACUUM then removes the old small files that are no longer referenced (after the retention period). Run OPTIMIZE first because VACUUM deletes the old files OPTIMIZE replaces.

Q125

What does 'Out of Memory' error on the Spark driver (not executor) typically indicate?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. The cluster has too few workers
- B. The driver is collecting too much data locally (e.g. df.collect() on a large DataFrame) or a broadcast table is too large ✓
- C. The Delta table is corrupted
- D. The JDBC connection dropped

Answer: B | **Why:** Driver OOM typically occurs when: (1) df.collect() brings all data to driver RAM, (2) a broadcast table exceeds spark.driver.maxResultSize, or (3) too many results are returned to the driver. Fix: avoid collect() on large data, filter before collect, or increase spark.driver.memory.

Q126

What does the SQL/DataFrame tab in the Spark UI show?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. SQL query history
- B. Physical execution plan as a visual DAG with stage boundaries, exchange (shuffle) nodes, join strategies, and row count statistics at each operation ✓
- C. Database schema browser
- D. Active SQL connections

Answer: B | **Why:** The SQL/DataFrame tab shows the physical query plan as a visualised DAG. It reveals: scan operations (how many files skipped), exchange/shuffle nodes, join strategies (broadcast vs sort-merge), and row counts at each stage — essential for query optimisation.

Q127

A streaming pipeline shows processing latency increasing over time. Which Spark UI metric should be checked first?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. GC time on the driver
- B. Batch processing time in the Streaming tab — if it exceeds the trigger interval, the pipeline is falling behind ✓
- C. Shuffle read size
- D. Number of input files

Answer: B | **Why:** In the Streaming tab of Spark UI, batch processing time shows how long each micro-batch takes. If batch processing time > trigger interval, the pipeline is falling behind (processing slower than data arrives). Solutions: increase parallelism, optimise transformations, or scale up cluster.

Q128

A cluster init script fails silently and the cluster starts but packages are missing. How should the data engineer debug this?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. Check the cluster's Python version
- B. View the init script logs in the cluster event log and driver/executor log directories (usually /databricks/init_scripts/) ✓**
- C. Reinstall all packages manually
- D. Restart the cluster multiple times

Answer: B | **Why:** Init script failures are logged in the cluster event log (Clusters → Events tab) and to /databricks/init_scripts//.log on the driver. These logs show exactly where the script failed. Use 'set -e' in bash init scripts to fail fast on errors.

Q129

What is the advantage of Liquid Clustering over traditional PARTITION BY in Delta Lake?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. Liquid Clustering uses less storage
- B. PARTITION BY creates static directory-based partitions (hard to change); Liquid Clustering allows flexible cluster key changes without full table rewrites and handles high-cardinality columns better ✓**
- C. Liquid Clustering is faster than partitioning by 10x
- D. PARTITION BY doesn't work with Delta Lake

Answer: B | **Why:** PARTITION BY creates physical directory structure per distinct value (e.g. date=2026-01-01/). Changing partition keys requires full table rewrite. High cardinality columns create too many small partitions. Liquid Clustering stores clustering metadata separately, handles high cardinality, and allows key changes without rewriting.

Q130

A Lakeflow Job shows status 'BLOCKED' for several tasks. What does this mean?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. The tasks are queued due to insufficient cluster capacity
- B. The tasks are blocked because one or more upstream dependency tasks have not yet completed successfully ✓**
- C. The tasks have exceeded their timeout
- D. The cluster was shut down

Answer: B | **Why:** BLOCKED status means the task is waiting for upstream dependencies. The task won't start until all tasks it depends on (with ALL_SUCCESS or appropriate condition) complete successfully. Check the DAG view to identify which upstream task is running or failed.

Q131

Which Spark tuning parameter controls whether Spark automatically broadcasts small tables in join operations?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. spark.sql.join.broadcastEnabled
- B. spark.sql.autoBroadcastJoinThreshold (default 10MB — tables smaller than this are auto-broadcast) ✓**
- C. spark.broadcast.maxTableSize
- D. spark.join.strategy=broadcast

Answer: B | **Why:** spark.sql.autoBroadcastJoinThreshold (default 10MB) controls automatic broadcast join. Tables smaller than this threshold are automatically broadcast to all executors. Set to -1 to disable auto-broadcast. Increase (e.g. 100MB) for larger lookup tables that fit in executor memory.

Q132

What is the recommended approach for a Delta table used heavily for point lookups on a user_id column with 10 million distinct values?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. PARTITION BY user_id (creates 10M partitions — catastrophic)
- B. Enable Liquid Clustering with CLUSTER BY (user_id) — handles high cardinality efficiently with incremental reclustering ✓**
- C. Add a secondary index
- D. Use Z-Order with partitioning on date + Z-Order on user_id

Answer: B | **Why:** PARTITION BY on a 10M cardinality column creates 10M directories — catastrophic small file problem. Liquid Clustering with CLUSTER BY (user_id) handles high-cardinality columns by co-locating rows with similar user_ids in the same files without creating millions of directories.

Q133

A data engineer wants to identify which stage in a Spark job is taking the longest. Where should they look?

Domain 6: Troubleshooting, Monitoring & Optimization

- A. The Job summary page
- B. The Stages tab in Spark UI — shows duration, task metrics, shuffle read/write, and input/output size per stage ✓**
- C. The Executors tab
- D. The Environment tab

Answer: B | **Why:** The Stages tab shows each stage with: total duration, number of tasks, shuffle read/write bytes, input/output records, and individual task-level metrics. Sort by Duration descending to immediately see the bottleneck stage. Click a stage to see task-level details and identify skew.

Q134

What is the impact of setting `spark.sql.shuffle.partitions` too high for a small dataset?

Domain 6: Troubleshooting, Monitoring & Optimization

A. Improved performance due to higher parallelism

B. Thousands of tiny tasks with more scheduling overhead than actual computation — wasting time and slowing the job down ✓

C. No impact — Spark handles it automatically

D. The job fails with too many partitions error

Answer: B | Why: For a 100MB dataset, 1000 shuffle partitions creates 1000 tasks each processing ~100KB. Task scheduling overhead (launching JVM, serialisation, network) exceeds computation time. This creates the 'too many small tasks' antipattern. Reduce to 8-16 for small datasets.

★ Domain 7: Governance & Security

Q135

Which SQL command grants read-only access to all tables in a schema to an analyst group?

Domain 7: Governance & Security

A. GRANT ALL PRIVILEGES ON SCHEMA sales_data TO analysts

B. GRANT SELECT ON SCHEMA sales_data TO analysts ✓

C. GRANT INSERT ON SCHEMA sales_data TO analysts

D. GRANT SELECT ON ALL TABLES IN SCHEMA sales_data TO analysts

Answer: B | Why: GRANT SELECT ON SCHEMA schema_name TO group grants SELECT on all current AND future tables within that schema to the specified principal. This is more efficient than granting SELECT on each table individually. Assumes USE CATALOG and USE SCHEMA are already granted.

Q136

A data engineer creates a table with `CREATE TABLE t (id INT) LOCATION 'abfss://...'`. What type of table is this?

Domain 7: Governance & Security

A. Managed table

B. External table ✓

C. Temporary table

D. Delta view

Answer: B | Why: Specifying LOCATION creates an external table. Unity Catalog manages the metadata but the data files reside at the external path. Dropping the table removes only the UC metadata — data files at the LOCATION remain intact. Without LOCATION, a managed table is created.

Q137

What is the minimum set of UC privileges required for a user to query a table?

Domain 7: Governance & Security

A. SELECT on the table only

B. ALL PRIVILEGES on the catalog

C. USE CATALOG on the catalog + USE SCHEMA on the schema + SELECT on the table ✓

D. OWNER of the catalog

Answer: C | Why: UC uses layered access: USE CATALOG (traverse the catalog level), USE SCHEMA (traverse the schema level), and SELECT (read the table data). Missing any one of these prevents access. USE CATALOG + USE SCHEMA don't grant data access — SELECT is additionally required.

Q138

What is the difference between REVOKE and DENY in Unity Catalog?

Domain 7: Governance & Security

A. They are identical

B. REVOKE removes a previously granted privilege; DENY explicitly blocks a privilege — overrides GRANT even if the user has it via group membership ✓

C. REVOKE is for users; DENY is for groups

D. DENY is for column-level; REVOKE is for table-level

Answer: B | Why: REVOKE removes a GRANT — if the user has the privilege via another grant (group membership), they still have access. DENY explicitly blocks access regardless of other grants — if a user is both in a group with GRANT and has a DENY, DENY wins. Use DENY to override inherited permissions.

Q139

A company needs to ensure sensitive salary data in an HR table is hidden from all users except HR managers, who see full values. Which UC feature implements this?

Domain 7: Governance & Security

- A. Row-level security filter
- B. Column masking — applies a SQL function that returns NULL (or masked value) for non-HR users and the actual value for HR managers ✓**
- C. DENY SELECT on the salary column
- D. Encrypt the column in storage

Answer: B | **Why:** Column masking applies a user-defined SQL function at query time: IF(is_member('HR_managers'), salary, NULL). Non-HR users see NULL in the salary column; HR managers see actual values. The function receives the column value and returns a masked or unmasked result.

Q140

A data team wants only users in the 'finance_team' group to see rows where department = 'Finance' in a shared table. Which UC feature achieves this?

Domain 7: Governance & Security

- A. Column masking on the department column
- B. Row-level security (row filter) — a boolean function applied at query time that returns true only for rows matching the user's group ✓**
- C. Table-level GRANT with WHERE clause
- D. Partitioning by department and granting per-partition access

Answer: B | **Why:** Row filters apply a row-filtering function: is_member('finance_team') AND department = 'Finance' OR NOT is_member('finance_team') AND department != 'Finance'. Applied at query time, users only see rows their filter allows. Implemented as a UC ABAC policy or row filter function.

Q141

What is Unity Catalog ABAC (Attribute-Based Access Control)?

Domain 7: Governance & Security

- A. A Databricks billing model based on attribute usage
- B. A centralised policy model combining row filters and column masks based on user attributes (group membership, custom tags) to control fine-grained data access across multiple tables ✓**
- C. An access control system for Databricks clusters only
- D. A feature for controlling API access rates

Answer: B | **Why:** ABAC policies in UC define fine-grained access rules based on subject attributes (who: user groups, service principals) and object attributes (what: table tags, column sensitivity labels). These policies centrally enforce row/column-level security consistently across all tables matching the policy scope.

Q142

How does Unity Catalog lineage work?

Domain 7: Governance & Security

- A. Engineers manually document lineage in a separate metadata tool
- B. UC automatically instruments query execution to capture column-level lineage across notebooks, SQL queries, jobs, and pipelines — viewable in Catalog Explorer without any code changes ✓**
- C. Lineage requires special annotations in notebook code
- D. Lineage only tracks table-level, not column-level dependencies

Answer: B | **Why:** UC auto-captures data lineage as queries run. It records: which columns read from which tables, written to which tables, across all Databricks surfaces (notebooks, jobs, SQL Warehouses, pipelines). Column-level lineage is shown in Catalog Explorer — no code instrumentation required.

Q143

What is Delta Sharing and how does it benefit data sharing?

Domain 7: Governance & Security

- A. A feature to copy Delta tables to other Databricks workspaces
- B. An open protocol that shares live Delta table data with external recipients without data copying — recipients always see current data, using open-source client libraries, no Databricks account needed ✓**
- C. A backup mechanism for Delta tables
- D. A feature to sync Delta tables across cloud regions

Answer: B | **Why:** Delta Sharing provides live, zero-copy data sharing. The provider creates a share, adds tables, creates a recipient with a token. Recipients use open-source clients (Python/pandas/Spark/Power BI) to read live data. No data duplication, no stale copies, no Databricks dependency for recipients.

Q144

What is Lakehouse Federation in Unity Catalog?

Domain 7: Governance & Security

A. Federating multiple Databricks workspaces under one governance layer

B. Creating foreign catalogs in UC that map to external database systems (MySQL, PostgreSQL, Snowflake) enabling direct SQL queries without copying data into Databricks ✓

C. Sharing data between different cloud providers

D. Federating authentication across identity providers

Answer: B | **Why:** Lakehouse Federation creates a foreign catalog in UC pointing to an external data source. Users query it with standard SQL from Databricks as if it were a native UC table. Data is queried in place — no ETL, no data movement. Useful for federated queries across heterogeneous data sources.

Q145

Where are Unity Catalog audit logs stored?

Domain 7: Governance & Security

A. In a special audit table in every UC schema

B. Delivered to customer-owned cloud storage (S3/ADLS/GCS) as JSON files via Databricks diagnostic log delivery ✓

C. In the Databricks control plane database only

D. In DBFS at /audit-logs/

Answer: B | **Why:** Databricks workspace audit events (logins, data access, permission changes, job runs) are delivered to customer-configured cloud storage as JSON files. Customers own and control the logs. These can be queried with Databricks SQL or ingested into SIEM tools for security monitoring.

Q146

A data engineer drops a managed Delta table in Unity Catalog. Can the data be recovered?

Domain 7: Governance & Security

A. Yes — always, because UC keeps a 90-day recycle bin

B. Yes — if VACUUM hasn't yet deleted the data files and Delta time travel retention hasn't expired, an admin can recreate the table at the same location to access historical versions ✓

C. No — managed table data is permanently and immediately deleted

D. Yes — data is automatically backed up to DBFS

Answer: B | **Why:** Managed table DROP deletes data, but if executed recently, the Parquet files may still exist (VACUUM hasn't run). An admin can access Delta time travel by recreating the table or directly reading the Delta log path. Once VACUUM runs past the retention period, data is permanently gone.

Q147

Which SQL command creates a Delta Sharing share?

Domain 7: Governance & Security

A. CREATE SHARE my_share ✓

B. INIT SHARE my_share

C. NEW SHARE my_share USING DELTA

D. CREATE DELTA SHARE my_share

Answer: A | **Why:** CREATE SHARE share_name is the SQL DDL command to create a new Delta Share object in Unity Catalog. Then: ALTER SHARE my_share ADD TABLE catalog.schema.table; CREATE RECIPIENT recipient_name; GRANT SELECT ON SHARE my_share TO RECIPIENT recipient_name.

Q148

What limitation does Delta Sharing impose on recipients?

Domain 7: Governance & Security

A. Recipients can only access data once per day

B. Recipients can only read data — Delta Sharing is read-only; recipients cannot modify, update, or delete rows in shared tables ✓

C. Recipients need a Databricks account

D. Recipients can only access the last 7 days of data

Answer: B | **Why:** Delta Sharing is a read-only protocol for recipients. The data provider retains full write control. Recipients can query and download data (subject to their permissions) but cannot write back. This ensures data providers maintain full control over their data assets.

Q149

What is the cost consideration for Delta Sharing across different cloud providers?

Domain 7: Governance & Security

- A. Delta Sharing across clouds is free — it's a zero-copy protocol
- B. Cross-cloud data access incurs cloud egress fees as data must traverse the internet between cloud provider networks — potentially significant for large datasets ✓
- C. Delta Sharing requires purchasing a cross-cloud license
- D. Only the recipient pays — the provider has no cost

Answer: B | **Why:** When a recipient on Azure accesses data shared from AWS (or vice versa), the AWS provider pays egress fees for data leaving their network. For large datasets, this can be substantial. Cost-conscious architectures prefer same-cloud, same-region sharing to minimise or eliminate egress costs.

Q150

Which statement about converting a managed table to an external table in Unity Catalog is correct?

Domain 7: Governance & Security

- A. You cannot convert between managed and external tables
- B. Use `CREATE TABLE new_ext_table LOCATION 'path' AS SELECT * FROM managed_table`, then drop the managed table ✓
- C. `ALTER TABLE managed_table CONVERT TO EXTERNAL LOCATION 'path'`
- D. Managed and external tables are determined at creation — conversion is not possible after

Answer: B | **Why:** To convert managed to external: (1) `CREATE TABLE ext_table LOCATION 'path' AS SELECT * FROM managed_table` (copies data to external path), (2) `DROP TABLE managed_table` (deletes managed copy). There's no direct `ALTER TABLE` conversion. In Unity Catalog, data must be explicitly moved to the external location.

Q151

A service principal needs to read data from a UC table for an automated pipeline. Which SQL grants the minimum required permissions?

Domain 7: Governance & Security

- A. `GRANT ALL PRIVILEGES ON CATALOG my_catalog TO my_service_principal`
- B. `GRANT USE CATALOG ON CATALOG my_catalog TO my_service_principal; GRANT USE SCHEMA ON SCHEMA my_schema TO my_service_principal; GRANT SELECT ON TABLE my_table TO my_service_principal` ✓
- C. `GRANT OWNER ON TABLE my_table TO my_service_principal`
- D. `GRANT SELECT ON TABLE my_table TO my_service_principal`

Answer: B | **Why:** Principle of least privilege: service principals need exactly `USE CATALOG` + `USE SCHEMA` + `SELECT` — no more. `GRANT ALL` or `OWNER` grants unnecessary write/admin capabilities. Option D (`SELECT` only) would fail because `USE CATALOG` and `USE SCHEMA` are also required to access the table.

Q152

Which Unity Catalog feature automatically tracks how data in table A was derived from table B and column C?

Domain 7: Governance & Security

- A. Delta Change Data Feed
- B. Data lineage — UC auto-captures column-level provenance as queries run across notebooks, jobs, and SQL Warehouses ✓
- C. Audit logs
- D. Table statistics

Answer: B | **Why:** Unity Catalog data lineage automatically tracks data flow: which source columns were read to produce which target columns, across all Databricks workloads. This is captured without code changes and displayed in Catalog Explorer, enabling impact analysis and data governance compliance.

Q153

What does granting `USE CATALOG` on a catalog allow?

Domain 7: Governance & Security

- A. Full read/write access to all objects in the catalog
- B. The ability to traverse the catalog and see schema names — it does NOT grant read access to any tables or data ✓
- C. The ability to create new schemas
- D. Administrative control over the catalog

Answer: B | **Why:** `USE CATALOG` allows a principal to see and navigate into the catalog. It's a prerequisite for any other action within the catalog but grants NO data access by itself. You still need `USE SCHEMA` (to navigate schemas) and `SELECT` (to read data) as additional grants.

Q154

A data engineer needs to share a Delta table with an external partner using Delta Sharing. The partner uses Apache Spark (not Databricks). What does the partner need to access the data?

Domain 7: Governance & Security

- A. A Databricks account and workspace
- B. The open-source delta-sharing Spark connector, a sharing endpoint URL, and an access token provided by the data provider ✓**
- C. A Unity Catalog metastore
- D. Direct access to the provider's cloud storage bucket

Answer: B | **Why:** External (non-Databricks) recipients use: (1) delta-sharing open-source connector for their platform (Spark, pandas, Power BI), (2) a profile file containing endpoint URL, bearer token, and expiration. No Databricks account, Unity Catalog, or storage access required — data is served through the Delta Sharing server.

Q155

When using Lakehouse Federation, where does query processing happen?

Domain 7: Governance & Security

- A. All processing happens in the external database
- B. All processing happens in Databricks on the fetched data
- C. Processing is split: query pushdown to external DB for filtering, with remaining processing in Databricks ✓**
- D. Lakehouse Federation only works with Databricks-to-Databricks connections

Answer: C | **Why:** Lakehouse Federation uses predicate pushdown: filter conditions (WHERE clauses) are pushed to the external database to reduce data transferred to Databricks. Remaining transformations (joins with UC tables, aggregations not pushable to the source) execute in Databricks Spark.

Q156

Which statement is correct about Unity Catalog privilege inheritance?

Domain 7: Governance & Security

- A. Privileges must be explicitly granted on every individual table — no inheritance
- B. Granting a privilege on a catalog automatically applies it to all current AND future schemas and tables within that catalog ✓**
- C. Privileges only inherit downward from schema to table, not from catalog to schema
- D. Inheritance only applies to SELECT, not to CREATE or MODIFY

Answer: B | **Why:** UC privilege inheritance works through all three levels: GRANT SELECT ON CATALOG c → applies to all schemas and tables in c (current and future). GRANT SELECT ON SCHEMA s → applies to all tables in s. This enables efficient bulk permission management without per-table grants.